

11

ACTIVE DIRECTORY



The previous chapter described the authentication configuration of a local domain. In this chapter, we'll detail how Active Directory stores user and group configurations on an enterprise network domain. We'll begin by inspecting the domain configuration, using various PowerShell commands that can enumerate the configured trust relationships, users, and groups. We'll then dig into the structure of Active Directory and how you can access its raw information over the network.

Once you understand how Active Directory is structured, we'll explore how Windows determines who can inspect and modify it. As you'll see, like most Windows platforms, Active Directory uses security descriptors to grant or deny access to the configuration.

A Brief History of Active Directory

Prior to Windows 2000, the user configuration for an enterprise network was stored in a SAM database on the network's domain controller. The domain controller authenticated users with the Netlogon protocol, which relied on the MD4 password hash format. To modify the SAM database, you could use the SAM remote service, as described in the previous chapter. This service allowed an administrator to add or remove users and groups on the domain controller.

As enterprise networks became more complex, the SAM database format proved to be limited. Windows 2000, which overhauled enterprise networking, moved the user configuration to Active Directory and changed the primary authentication protocol from Netlogon to Kerberos.

Active Directory provides several advantages over the SAM database, as it is extensible and can store arbitrary data. For example, an administrator can store additional information with a user's configuration to represent their security clearance, and an application can check this information when granting or denying access to a resource. Active Directory also has fine-grained security, allowing administrators to delegate parts of the configuration to different users more easily than the SAM can.

Active Directory is stored locally on a domain controller, and computers in the network can access it using the *Lightweight Directory Access Protocol (LDAP)*, which exposes a TCP/IP network connection on port 389. LDAP derives from the more complex *Directory Access Protocol (DAP)*, which formed part of the X.500 directory service specification. If you're familiar with the X.509 certificate format for exchanging public key information on secure websites, some of the following concepts might seem familiar.

Exploring an Active Directory Domain with PowerShell

Let's begin our exploration of Active Directory with a high-level look at a domain configuration. **Figure 11-1** shows an example forest (of course, your configuration might differ).

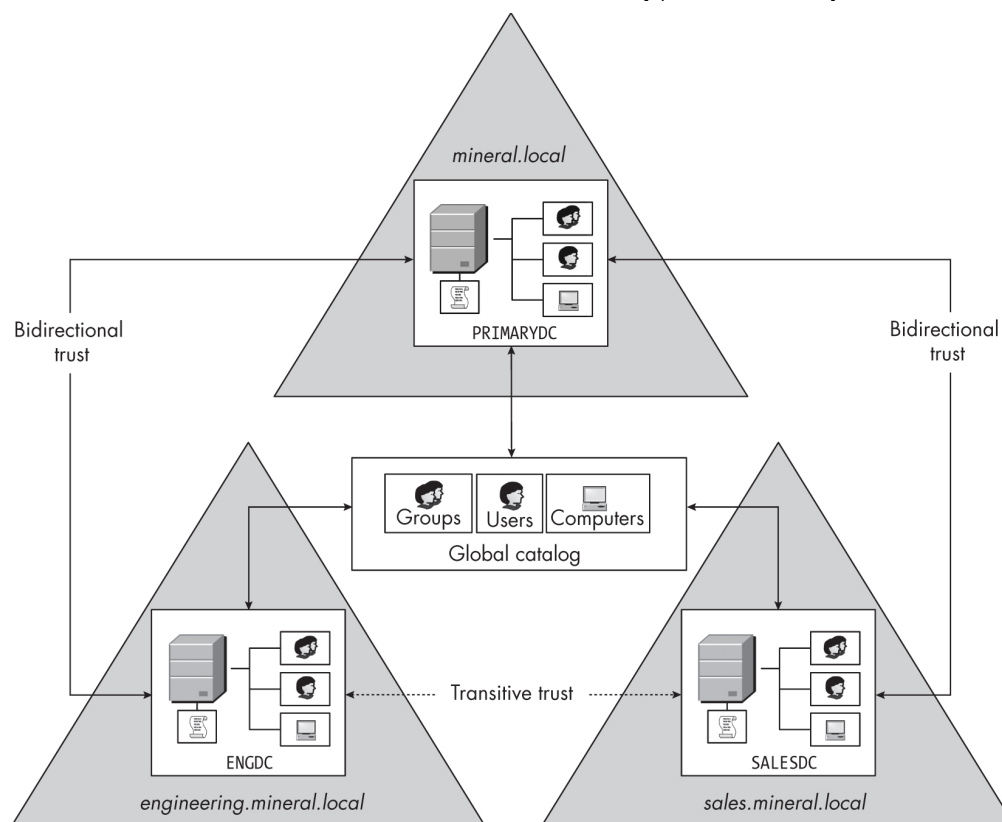


Figure 11-1: An example Windows forest

To explore this forest, we'll run various PowerShell commands that can enumerate its domains, users, groups, and devices. If you'd like to follow along, you can find setup instructions for a similar domain configuration in [Appendix A](#).

The Remote Server Administration Tools

We can interact with the Active Directory server through PowerShell's `ActiveDirectory` module, which ships with the optional Remote Server Administration Tools (RSAT) Windows capability. By default, only domain controllers come with RSAT installed, as the commands are designed for managing the directory (which not every client system needs to do).

Therefore, you might need to install RSAT before running the example scripts in this chapter. If you're running a version of Windows older than Windows 10, version 1809, you must download RSAT from the Microsoft website. If you're using a newer version of Windows, you can install RSAT by running the commands in [Listing 11-1](#) from an administrator PowerShell console.

```
PS> $cap_name = Get-WindowsCapability -Online |
Where-Object Name -Match 'Rsat.ActiveDirectory.DS-LDS.Tools'
PS> Add-WindowsCapability -Name $cap_name.Name -Online
```

Listing 11-1: Installing the Remote Server Administration Tools

Note that the examples in this section won't work unless you run the commands on a machine joined to a Windows enterprise network, such as the one described in [Appendix A](#).

Basic Forest and Domain Information

Let's start by gathering some basic information about the forest and domain we're connected to. You can follow along by executing the commands in [Listing 11-2](#) on a computer in the root *mineral.local* domain of the example forest.

```
❶ PS> $forest = Get-ADForest
❷ PS> $forest.Domains
mineral.local
sales.mineral.local
engineering.mineral.local

❸ PS> $forest.GlobalCatalogs
PRIMARYDC.mineral.local
SALESDC.sales.mineral.local
ENGDC.engineers.mineral.local

❹ PS> Get-ADDomain | Format-List PDCEmulator, DomainSID, DNSRoot, NetBIOSName
PDCEmulator : PRIMARYDC.mineral.local
DomainSID   : S-1-5-21-1195776225-522706947-2538775957
DNSRoot     : mineral.local
NetBIOSName : MINERAL

❺ PS> Get-ADDomainController | Select-Object Name, Domain
Name      Domain
----      -
PRIMARYDC mineral.local

❻ PS> Get-ADTrust -Filter * | Select-Object Target, Direction, TrustType
Target                Direction      TrustType
-----
-----
```

engineering.mineral.local	BiDirectional	Uplevel
sales.mineral.local	BiDirectional	Uplevel

Listing 11-2: Listing some basic information about the forest and domain

We first request information about the current forest using the `Get-ADForest` command ❶. The returned object has many properties, but here we focus on two of them. The `Domains` property returns a list of the Domain Name System (DNS) names for the domains in the forest ❷. In this example, it matches the forest in **Figure 11-1**. We also inspect the `GlobalCatalogs` property, which lists all systems that maintain a copy of the shared global catalog ❸. We can use these to inspect the forest-level configuration.

We then run the `Get-ADDomain` command, which returns information about the domain to which the current system is connected ❹. Here, we select four properties. The first one, `PDCemulator`, is the DNS name of the *primary domain controller (PDC) emulator*. The PDC, which used to be the main domain controller in the local domain, once acted as the definitive user database. (A backup domain controller served as a secondary database, in case the PDC went down.) With the introduction of Active Directory, it became possible to more evenly distribute the authentication workload without the PDC. However, Windows still gives the PDC emulator preferential treatment; for example, when you change your password, the operating system will always first try to change it on the PDC. The PDC also runs the legacy Netlogon service, for backward compatibility with older versions of Windows.

The next property is the `DomainSID`. This SID serves as the basis for all other user and group SIDs in the domain. It's equivalent to the machine SID we saw in **Chapter 10**, but it applies to the entire network. The final two properties are the `DNSRoot` and `NetBIOSName`. These are the domain's root DNS name and simple domain name, which Windows keeps around for legacy support reasons.

A good example of this legacy support involves the names of users in a domain. Officially, you should refer to users with a fully qualified name, the *user principal name (UPN)*, which takes the form `alice@mineral.local`. However, in the user interface you use to log in to your computer, you typically won't enter the UPN as your username; instead, you'd enter

something like *MINERAL|alice*, which we refer to as a *down-level logon name*.

Next, we list the domain controllers on the domain the system is connected to using the `Get-ADDomainController` command ⑤. We're inspecting a simple domain, so [Listing 11-2](#) contains only a single entry, `PRIMARYDC`. As we saw earlier, though, the forest contains multiple domains. We can enumerate the configured trust relationships using the `Get-ADTrust` command ⑥. The output reveals all of the trusts to be bidirectional. The third column identifies the type of each domain: `Up1evel` indicates that the domain is also based on Active Directory, while a value of `Down1evel` would represent a pre-Windows 2000 domain.

The Users

Let's now list the user account information stored on the Active Directory server. We can do this with the `Get-ADUser` command, as shown in [Listing 11-3](#).

```
PS> Get-ADUser -Filter * | Select-Object SamAccountName, Enabled, SID
SamAccountName Enabled SID
-----
Administrator    True S-1-5-21-1195776225-522706947-2538775957-500
Guest             False S-1-5-21-1195776225-522706947-2538775957-501
krbtgt            False S-1-5-21-1195776225-522706947-2538775957-502
bob               True S-1-5-21-1195776225-522706947-2538775957-1108
alice             True S-1-5-21-1195776225-522706947-2538775957-1110
```

Listing 11-3: Displaying the Active Directory server's users

Using `Get-ADUser` is like using `Get-LocalUser`, except that you need to specify a filter. In [Listing 11-3](#) we specify `*` to get all users, but on a real network you'll find filtering important to reduce the output, as the Active Directory server could contain hundreds or thousands of users.

The output shows each user's plain username (in the `SamAccountName` column), whether the user is enabled, and their SID. As with the local users, each SID has a common prefix that should match the domain SID from [Listing 11-2](#).

The user's password is stored in a special write-only attribute in the Active Directory server. We can't read this password from outside the domain controller except via backups of the directory or when the directory is replicated between domain controllers.

The Groups

To list the security groups in the Active Directory server, we can use the `Get-ADGroup` Command ([Listing 11-4](#)).

```
PS> Get-ADGroup -Filter * | Select-Object SamAccountName, SID, GroupScope
SamAccountName  SID                                     GroupScope
-----
Administrators   S-1-5-32-544                         DomainLocal
Users           S-1-5-32-545                         DomainLocal
Guests          S-1-5-32-546                         DomainLocal
--snip--
Enterprise Admins S-1-5-21-1195776225-522706947-2538775957-519 Universal
Cert Publishers  S-1-5-21-1195776225-522706947-2538775957-517 DomainLocal
Domain Admins    S-1-5-21-1195776225-522706947-2538775957-512 Global
Domain Users     S-1-5-21-1195776225-522706947-2538775957-513 Global
--snip--
```

Listing 11-4: Displaying the Active Directory server's groups

Notice that the output includes both *BUILTIN* groups, such as *Administrators*, and domain groups, such as *Enterprise Admins*. You can easily distinguish these group types based on the domain SID used as the prefix of a group's SID. In this example, the domain SID prefix is `S-1-5-21-1195776225-522706947-2538775957`.

The system uses the *BUILTIN* groups only when a user authenticates to the domain controller. For example, adding a user to the *BUILTIN\Administrators* group would grant that user administrator access to the database on the domain controller, but not on any other machine in the network. On the other hand, the domain groups get added to the user's token when they authenticate, and they can be used for access checks on the local computer.

Domain groups can have three possible scopes. The `Global` group scope is visible to the entire forest. While any domain in the forest can use the group, it contains users or groups in the defining domain only. A `Global` group is equivalent to the group object in the SAM configuration we covered in the previous chapter. By contrast, a `DomainLocal` group is visible only in the defining domain, but it can contain any user or group from any trusted domain. It's equivalent to the alias object in the SAM database.

The `Universal` group scope combines the global visibility and broad membership of the two other scopes: groups in this scope are visible to the entire forest and can contain any user or group.

To highlight the distinction between the `Universal` and `Global` group scopes, let's consider the difference between two groups, *Enterprise Admins* and *Domain Admins*. *Enterprise Admins* includes all the users who can manage a forest. While there should be only one instance of this group, defined in the root domain, you might want to be able to add any user across the forest as a member. Therefore, as you can see in [Listing 11-4](#), it's a `Universal` group. All domains can use it, and it can contain anyone.

In contrast, *Domain Admins* contains users who are administrators of a single domain. Other domains might use the group as a resource if it is configured to grant them access, but it restricts its membership to the defining domain. Therefore, it's a `Global` group. If you're managing only a single domain, the differences between these scopes aren't particularly relevant.

The SAM remote service would return `DomainLocal` groups when you enumerate alias objects and both `Universal` and `Global` groups when you enumerate group objects. You might find it odd that the service returns `Universal` groups as group objects; after all, the APIs used to manipulate group object members allow you to specify a member using the domain's relative ID only, preventing you from using the SAM remote service to modify a `Universal` group if it has any members outside of the domain. In any case, you shouldn't really use the SAM remote service to manage an Active Directory domain.

You can list the members of an Active Directory server group using the `Get-ADGroupMember` command, as shown in [Listing 11-5](#).


```

PS> Get-ADGroupMember -Identity Administrators | Select Name, objectClass
Name                                objectClass
----                                -
Domain Admins                      group
Enterprise Admins                  group
Administrator                      user

PS> Get-LocalGroupMember -Name Administrators
ObjectClass Name                                PrincipalSource
-----
Group       MINERAL\Domain Admins                  ActiveDirectory
User        MINERAL\alice                                ActiveDirectory
User        GRAPHITE\admin                              Local
User        GRAPHITE\Administrator                      Local

```

Listing 11-5: Displaying Administrators group members once they've joined the domain

Here, we enumerate the members of the *BUILTIN\Administrators* group on the domain controller. Because this is a *BUILTIN* group, users receive membership to the group only once they've authenticated to the domain controller.

However, when you join a computer to a domain, you can modify the local groups on that computer to include domain groups. For example, when we use `Get-LocalGroupMember` to list the members of the local *BUILTIN\Administrators* group, we see that the *Domain Admins* group from the domain has been added as a member. This change allows all administrators in the domain to be local administrators on any computer in the domain.

The Computers

When you join a computer to a domain, an account is created in the domain. These special user accounts grant the computer access to certain domain services before any user has authenticated to the system. The computer account is especially important for configuring the group policy, as well as for authenticating users to the system, as we'll see in

[Chapter 14.](#)

You can list the computer accounts on the Active Directory server using the `Get-ADComputer` command, shown in [Listing 11-6](#).

```
PS> Get-ADComputer -Filter * | Select-Object SamAccountName, Enabled, SID
SamAccountName Enabled SID
-----
PRIMARYDC$      True S-1-5-21-1195776225-522706947-2538775957-1000
GRAPHITE$        True S-1-5-21-1195776225-522706947-2538775957-1104
CINNABAR$        True S-1-5-21-1195776225-522706947-2538775957-1105
TOPAZ$           True S-1-5-21-1195776225-522706947-2538775957-1106
PYRITE$          True S-1-5-21-1195776225-522706947-2538775957-1109
HEMATITE$        True S-1-5-21-1195776225-522706947-2538775957-1113
```

Listing 11-6: Displaying the computer account SIDs

As this output shows, the computer account names usually have a trailing dollar sign character (\$), which makes it easy to differentiate computer accounts from user accounts. We can also see once again that the SIDs use the domain SID as a prefix. (The computers themselves continue to store their own separate machine SIDs in the local SAM database.)

A computer account needs a password to authenticate to the domain, and the domain-joined computer and domain controller automatically manage this password. By default, the computer generates a new complex password every 30 days and changes it on the domain controller. As the computer must change the password without user interaction, it stores the password in an LSA secret object called `$MACHINE.ACC`.

[Listing 11-7](#) shows how to query a computer's LSA secret using the `Get-LsaPrivateData` command. You'll need to run this command as an administrator. It's similar to the `Get-LsaSecret` command we saw in the previous chapter, except we don't need to manually open the policy and secret objects.

```
PS> Get-LsaPrivateData '$MACHINE.ACC' | Out-HexDump -ShowAll
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F - 0123456789ABCDEF
-----
00000000: 00 00 00 01 5F 5D 25 70 36 13 17 41 92 57 5F 50 - ...._]%p6..A.W_P
00000010: 89 EA AA 35 03 00 00 00 00 00 00 94 B1 CD 81 - ...5.....
```

```
00000020: 98 86 67 2A 31 17 1B E1 2F 5D 78 48 7B ED 0C 95 - ..g*1.../]xH{...
--snip--
```

Listing 11-7: Querying the \$MACHINE.ACC LSA secret

The LSA obfuscates the contents of the secret object, so just reading the value isn't enough to extract the password used for the computer account.

We've performed a high-level exploration of an Active Directory server configuration. Let's now look at how the directory is configured at a low level, so we can understand how it is secured.

Objects and Distinguished Names

Although we can use the commands in the `ActiveDirectory` module to access the user configuration, these commands hide the real structure of the Active Directory server, which consists of a hierarchical tree of entries, as shown in **Figure 11-2**.

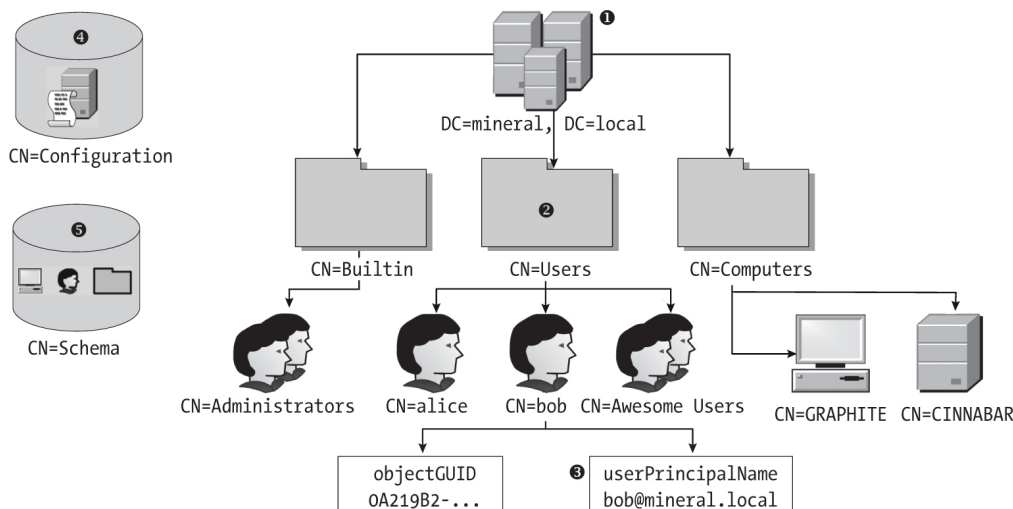


Figure 11-2: The structure of an Active Directory server

There are a few different types of entries, but the only ones we care about are *objects*, which store the user configuration. To refer to an object in the tree, we use its *distinguished name*, which must be unique across the directory. The distinguished name is a sequence of one or more *relative distinguished names* separated by commas. In the Active Directory server, you're most likely to encounter the following relative distinguished name types:

- c The country name
- cn The common name
- dc The domain component
- o The organization name
- ou The organizational unit name
- st The state or province name

For example, at the root of the directory is the domain object ❶, which has the domain name `DC=mineral,DC=local`. The `dc` relative distinguished name represents a domain component that is part of a DNS name. Taken together, this distinguished name represents the *mineral.local* DNS name of the domain.

Underneath the root object is a tree of objects that describe the configuration of the domain. I've shown only three of them in **Figure 11-2**. `CN` refers to a common name, a simple label for the object. The `CN=Users` object ❷ contains the user and group objects for the domain. The other two objects, `CN=Builtin` and `CN=Computers`, contain group accounts for the *BUILTIN* domain on the domain controller and the list of computer accounts, respectively.

To refer to the `Users` object, you would use its full distinguished name, `CN=Users,DC=mineral,DC=local`. Each user object could contain further objects, but it's more common for them to contain only a list of attribute values that represent the user's configuration ❸. For example, a user object might contain the `userPrincipalName` attribute, representing the UPN of the user in the Active Directory server.

Each object can also contain an `objectGUID` attribute with a GUID that uniquely identifies the object. Although unique, the distinguished name cannot consistently identify an object, as it would change if the object were moved or renamed. The `objectGUID` attribute stays the same even if the distinguished name changes.

Two separate root objects store administrative information for the domain root. These are the configuration object ❹ and the schema object ❺. The information stored in the configuration object matters to Active Directory security, and the schema object defines the directory's structure. We'll discuss both objects in more depth in later sections.

Enumerating Directory Objects

Default installations of the Active Directory server use well-known distinguished names, configurations, and schema objects. However, an administrator can change these names or add new directories to the database. For that reason, the Active Directory server exposes a special directory entry called the *Root Directory System Agent-Specific Entry (RootDSE)* that contains high-level configuration for the directory.

Listing 11-8 shows how to access the RootDSE entry for the current domain using the `Get-ADRootDSE` command.

```
PS> Get-ADRootDSE | Format-List '*NamingContext'
configurationNamingContext : CN=Configuration,DC=mineral,DC=local
defaultNamingContext        : DC=mineral,DC=local
rootDomainNamingContext    : DC=mineral,DC=local
schemaNamingContext        : CN=Schema,CN=Configuration,DC=mineral,DC=local
```

Listing 11-8: Inspecting the RootDSE entry for the current domain

As properties, we select the distinguished names for the *naming contexts*, which represent the top-level objects in the directory. Using these naming contexts, we can query objects on the Active Directory server with the `Get-ADObject` command (**Listing 11-9**).

```
❶ PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
❷ PS> Get-ADObject -SearchBase $root_dn -SearchScope OneLevel -Filter * |
Select-Object DistinguishedName, ObjectClass
DistinguishedName      ObjectClass
-----
❸ CN=Builtin,DC=mineral,DC=local      builtinDomain
CN=Computers,DC=mineral,DC=local      container
OU=Domain Controllers,DC=mineral,DC=local organizationalUnit
CN=ForeignSecurityPrincipals,DC=mineral,DC=local container
--snip--
```

```
❷ PS> Get-ADObject -Identity "CN=Builtin,$root_dn" | Format-List
DistinguishedName : CN=Builtin,DC=mineral,DC=local
Name              : Builtin
ObjectClass       : builtinDomain
ObjectGUID        : 878e2263-2496-4a56-9c6e-7b4db24a6bed

❸ PS> Get-ADObject -Identity "CN=Builtin,$root_dn" -Properties * | Format-List
CanonicalName      : mineral.local/Builtin
CN                 : Builtin
--snip--
```

Listing 11-9: Querying for the Active Directory server's objects

First we get the root domain naming context from the RootDSE ❶. This naming context represents the distinguished name for the directory's root domain object, which we can use to query for objects.

We then use the `Get-ADObject` command to query the child objects of the root ❷. The command takes various options to limit the scope of the child objects to return. The first is the `SearchBase` parameter, which returns only the children of a certain object (in this case, only the default naming context). We've supplied the default value here, which is unnecessary, but the parameter is useful in other cases.

The second option is the `SearchScope` parameter, which determines how recursive the search should be. We specify `OneLevel` to search only the immediate children of the search base. Other values include `Base`, which returns only the search base object, and `Subtree`, which recursively searches all child objects. The `Filter` parameter limits the values returned. In this case, we use `*` to return everything.

The output includes the `DistinguishedName` and `ObjectClass` attributes ❸. The `ObjectClass` attribute represents the name of the schema type, which we'll come back to in "The Schema" on page 353. We can select a specific distinguished name by specifying it as the value of the `Identity` parameter ❹. The object returned contains a list of the directory object's attributes as PowerShell properties. For example, we can see the `objectGUID` attribute, which represents the object's unique identifier.

In this case, the command returns only four values. For performance reasons, it queries for a small set of attributes, as some of the attribute values can be quite large. To query for more attributes, specify the `Properties` parameter, passing it either a list of attribute names or `*` to return all attributes 5.

Accessing Objects in Other Domains

What if you're on a computer in one domain of the forest and want to access the Active Directory server for another domain? You might attempt to use the distinguished name of the object you're interested in, as in

Listing 11-10.

```
PS> Get-ADObject -Identity 'CN=Users,DC=sales,DC=mineral,DC=local'
Get-ADObject : Cannot find an object with identity: 'CN=Users,DC=sales,DC=mineral,DC=local' under: 'DC=mineral,DC=local'.
```

Listing 11-10: Trying to access another domain's Active Directory

As you can see, trying to access an object in another domain's Active Directory server fails; the command tries to search for a child object with the specified distinguished name and can't find it.

To view the Active Directory server from another domain, you have a couple of options, shown in **Listing 11-11.**

```
PS> $dn = 'CN=Users,DC=sales,DC=mineral,DC=local'
❶ PS> $obj_sales = Get-ADObject -Identity $dn -Server SALES -Properties *
PS> $obj_sales.DistinguishedName
CN=Users,DC=sales,DC=mineral,DC=local

❷ PS> $obj_gc = Get-ADObject -Identity $dn -Server :3268 -Properties *
PS> $obj_gc.DistinguishedName
CN=Users,DC=sales,DC=mineral,DC=local

❸ PS> ($obj_sales | Get-Member -MemberType Property | Measure-Object).Count
28
PS> ($obj_gc | Get-Member -MemberType Property | Measure-Object).Count
25
```

Listing 11-11: Accessing the Active Directory server's objects in another domain

The first option is to explicitly specify the target domain using `Get-ADObject` with the `Server` parameter ❶. This parameter accepts the domain's simple name or DNS name, as well as the hostname of a domain controller within the domain. In this case, we specify `SALES`, and because this domain is part of our forest, the query returns a suitable domain controller.

The second option is to query the global catalog. As [Listing 11-2](#) showed, servers in the domain manage this catalog using data copied from other Active Directory servers. Select the global catalog by specifying the well-known port 3268 as the `Server` parameter ❷. In this example, we specify no domain or server name, which selects the global catalog in the current domain by default. If you wanted to, however, you could query the global catalog in another domain by prefixing the port with its name.

One thing to keep in mind is that the global catalog contains merely a subset of the full data in the Active Directory server. If we count the number of properties returned, we see that the object contains 28 properties ❸, whereas the global catalog version of it returns only 25. For certain object classes, the difference in property counts might be even more pronounced.

You might wonder: Why wouldn't you just query the domain directly for Active Directory information? Basically, it's a question of locality. The domain on which you're running the command might live on the other side of the world from the target domain, joined by a high-latency satellite link. Querying the target directly might be slow, expensive, or both. By contrast, the local global catalog might live on a domain controller in the next office, which offers convenience, even if it won't provide the same level of detail.

The Schema

The Active Directory server's schema describes the classes of object that exist, the attributes those classes might contain, and the relationships between classes. Each object in the directory is assigned to one or more classes; for example, a group is of the class `group`. You can find an object's class in its `objectClass` attribute.

Each object class has a corresponding schema type. The schema can organize these types in a hierarchy, as shown in **Figure 11-3**.

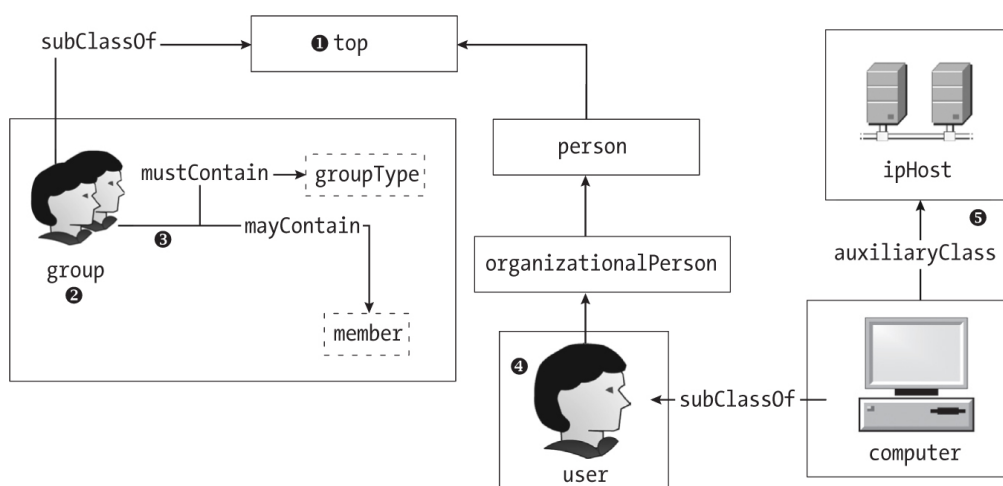


Figure 11-3: A schema hierarchy for the group, user, and computer classes

All schema class types derive from a base type, the top class ❶, and each class object's `subClassOf` attribute specifies the classes from which it derives. For example, the group type ❷ specifies top as its only `subClassOf` value.

Each class type can also include a list of the attributes that an instance of the class can contain ❸. This list is split into `mustContain`, for required attributes, and `mayContain`, for optional ones. In **Figure 11-3**, for example, the `mustContain` attribute has the required `groupType` attribute, used to indicate whether the group is Universal, Global, or DomainLocal. However, the `member` attribute, which contains the list of members of the group, is optional, as a group could have no members.

A second set of attribute lists, `systemMustContain` and `systemMayContain`, hold required and optional attributes that only the Active Directory server can modify; a normal user can't change these.

Not all class schema types are as simple as group. For example, the user class ❹ is a subclass of `organizationalPerson`, which itself is a subclass of `person`, which in turn is a subclass of top. Each of these schema class types can contribute required and optional attributes to the final subclass object.

A class can also contain lists of auxiliary classes, defined with the `auxiliaryClass` and `systemAuxiliaryClass` attributes ❺. We can use these classes to

add additional attributes to a schema class without making them part of the inheritance hierarchy.

Each class has an `objectClassCategory` attribute to define how the class can be used. It can be one of the following values:

Structural The class can be used as an object.

Abstract The class can be used for inheritance only.

Auxiliary The class can be used as an auxiliary only.

An additional type, `class88`, represents classes that were defined in the oldest LDAP specifications. Only certain system classes use this type, and new schema classes shouldn't use it.

Inspecting the Schema

We can inspect the schema using the same tools we would use to inspect user or group objects. An administrator can also modify the schema to add new types and attributes. For example, the Exchange mail server might modify the Active Directory server on which it's installed to add additional email address attributes for user objects.

As the schema is part of the directory, we can inspect it using the `Get-ADObject` command, as shown in [Listing 11-12](#).

```
❶ PS> $schema_dn = (Get-ADRootDSE).schemaNamingContext
PS> Get-ADObject -SearchBase $schema_dn -SearchScope OneLevel -Filter * |
Sort-Object Name | Select-Object Name, ObjectClass
Name                                ObjectClass
----                                -
❷ account                           classSchema
Account-Expires                     attributeSchema
Account-Name-History                attributeSchema
--snip--

❸ PS> Get-ADObject -SearchBase $schema_dn -Filter {
    ObjectClass -eq "classSchema"
} -Properties * | Sort-Object Name |
Format-List Name, {[guid]$_schemaIDGUID}, mayContain,
mustContain, systemMayContain, systemMustContain, auxiliaryClass,
```

```

systemAuxiliaryClass, SubClassOf
Name : account
❷ [guid]$_schemaIDGUID : 2628a46a-a6ad-4ae0-b854-2b12d9fe6f9e
❸ mayContain : {uid, host, ou, o...}
mustContain : {}
systemMayContain : {}
systemMustContain : {}
❹ auxiliaryClass : {}
systemAuxiliaryClass : {}
❺ SubClassOf : top
--snip--

❻ PS> Get-ADObject -SearchBase $schema_dn -Filter {
    LDAPDisplayName -eq "uid"
} -Properties * | Format-List adminDescription, {[guid]$_schemaIDGUID},
attributeSyntax, oMSyntax, oObjectClass
adminDescription : A user ID.
[guid]$_schemaIDGUID : 0bb0fca0-1e89-429f-901a-1413894d9f59
attributeSyntax : 2.5.5.12
oMSyntax : 64
oObjectClass :

```

Listing 11-12: Enumerating schema objects

We start by querying for all objects under the schema's naming context and displaying them to the shell ❶. The output shows the name of each schema object and its object class ❷. We can see two classes, `classSchema` and `attributeSchema`, which represent the schema types for object classes and attributes, respectively.

Next, we query for the schema objects and attributes again, but this time we use a filter to select only the objects whose `ObjectClass` attribute is equal to `classSchema` ❸. The `Filter` property takes a PowerShell-style expression that can filter the returned objects based on the object's attributes. The server evaluates this filter to improve performance, as it won't return objects that don't match the filter.

Note that the filter string isn't a full PowerShell script, even though it uses a similar syntax, so you can't perform complex scripting operations in the filter. The commands in the `ActiveDirectory` module also support the `LDAPFilter` parameter, which uses the LDAP specification's somewhat less intuitive filtering syntax. (Technically, even if you use the `Filter` param-

ter, PowerShell will convert it to an LDAP filter before sending the query to the LDAP server, as Active Directory doesn't yet execute PowerShell code directly.)

The returned class objects appear in the console, where I've highlighted some of their important attributes. The first is the `schemaIDGUID` attribute ④, which represents the unique identifier for the schema type. Microsoft documents most of these schema identifiers, although an administrator can also add their own. The directory stores the `schemaIDGUID` attribute as an array of bytes, so we convert it to a `guid` object to view the value more easily.

Note that the `schemaIDGUID` won't match the `objectGUID` attribute assigned to the object. The `objectGUID` should be unique in the directory, but it won't necessarily be unique globally. The `schemaIDGUID` should have the same value across all instances of the Active Directory server.

The next four attributes ⑤ represent the lists of attributes the class can contain. In this case, only `mayContain`, the list of optional class attributes, has any values. Each entry is identified by a name that is unique across the Active Directory server.

These lists are not exhaustive, however; in addition to these, the class could also incorporate attributes from its configured auxiliary classes (although in this example, none are listed ⑥). It will also incorporate any attributes inherited from the parent, which you can find in the `SubClassOf` attribute ⑦. To get the full list of attributes a class could contain, you need to enumerate the entire inheritance chain and all auxiliary classes.

Because it's unique, we can return an attribute's schema type by specifying a particular `LDAPDisplayName` attribute value. In this case, we use the first value in the attribute list, `uid` ⑧, and display a few of the schema type's attributes, including a description of the attribute and the `schemaIDGUID`.

Accessing the Security Attributes

As you just witnessed, manually inspecting the schema is a convoluted process. Still, we need to understand the schema to analyze the security

of the directory. For that reason, the `NtObjectManager` module comes with some commands that return the schema's security-specific attributes.

Listing 11-13 shows the simplest of these commands, `Get-DsSchemaClass`.

```
PS> Get-DsSchemaClass | Sort-Object Name
```

Name	SchemaId	Attributes
-----	-----	-----
account	2628a46a-a6ad-4ae0-b854-2b12d9fe6f9e	7
aCSPolicy	7f561288-5301-11d1-a9c5-0000f80367c1	17
aCSResourceLimits	2e899b04-2834-11d3-91d4-0000f87a57d4	5
aCSSubnet	7f561289-5301-11d1-a9c5-0000f80367c1	26
--snip--		

Listing 11-13: Enumerating all schema classes

When we specify no parameters, the command looks up all class type objects from the schema and returns them. The output shows each type's LDAP name and schema identifier, as well as the total number of attributes the type can contain, including all required and system attributes.

NOTE

Depending on the complexity of the schema and speed of the network, querying for all schema types can take a while. Once the command has downloaded the types, however, it will cache them, so you should receive a rapid response the next time you request them in the same PowerShell session.

Listing 11-14 shows how to inspect the `account` type using the module's commands.

```
PS> $cls = Get-DsSchemaClass -Name "account"
PS> $cls | Format-List
```

```
Name           : account
CommonName     : account
Description    : The account object class is used to define entries...
SchemaId       : 2628a46a-a6ad-4ae0-b854-2b12d9fe6f9e
SubClassOf    : top
Category       : Structural
Attributes     : {uid, host, ou, o...}
```

```

❶ PS> $cls.Attributes
Name           Required System
----
uid            False  False
host          False  False
ou            False  False
o             False  False
l             False  False
seeAlso       False  False
description    False  False

❷ PS> $cls.Attributes | Get-DsSchemaAttribute
Name           SchemaId                                     AttributeType
----
uid            0bb0fca0-1e89-429f-901a-1413894d9f59 String(Unicode)
host          6043df71-fa48-46cf-ab7c-cbd54644b22d String(Unicode)
ou            bf9679f0-0de6-11d0-a285-00aa003049e2 String(Unicode)
o             bf9679ef-0de6-11d0-a285-00aa003049e2 String(Unicode)
l             bf9679a2-0de6-11d0-a285-00aa003049e2 String(Unicode)
seeAlso       bf967a31-0de6-11d0-a285-00aa003049e2 Object(DS-DN)
description    bf967950-0de6-11d0-a285-00aa003049e2 String(Unicode)

❸ PS> Get-DsSchemaClass -Parent $cls -Recurse
Name SchemaId                                     Attributes
----
top   bf967ab7-0de6-11d0-a285-00aa003049e2 125

```

Listing 11-14: Inspecting a single class schema type

You can specify the name of the class using either the LDAP name with the `Name` parameter or the schema identifier with the `SchemaId` parameter.

The returned object contains an `Attributes` property, which holds the list of all attributes for the class ❶. Rather than including separate attribute lists, the command assigns each attribute the `Required` and `System` properties to indicate the list from which they were sourced.

To get more information about the attributes, you can pipe them into the `Get-DsSchemaAttribute` command, which looks up the schema attribute type ❷. This command returns the LDAP name (`Name`) and schema identifier (`SchemaId`) properties, as well as a decoded attribute type (`AttributeType`). We

can see, for example, that the `uid` type is a Unicode string, while the `seeAlso` type is a string that contains a distinguished name.

Finally, you can directly look up the parent class by using the `Parent` parameter and specifying the existing class object . You can also specify the `Recurse` parameter to recursively enumerate all parents. In this case, the only parent class is `top`, but querying a more complex class, such as `user`, would return several more schema classes.

Security Descriptors

Almost any time we must secure a resource in Windows, we'll turn to security descriptors and access checking, and with Active Directory it's no different. LDAP supports authentication, and the Active Directory server uses it to create a token that represents the user. It then uses this token to determine what objects and attributes a given user can manipulate. Let's begin by discussing how to query and store security descriptors on the Active Directory server.

Querying Security Descriptors of Directory Objects

Each directory object is assigned a security descriptor when it's created. The object stores this security descriptor as a byte array in a mandatory attribute named `nTSecurityDescriptor`. As this attribute is defined in the `top` class, all object classes require it. [Listing 11-15](#) checks the attribute schema class and shows that `Required` is `True`.

```
PS> (Get-DSSchemaClass top).Attributes |
Where-Object Name -Match nTSecurityDescriptor
Name                               Required System
----                               -
nTSecurityDescriptor              True      True
```

Listing 11-15: Checking the `nTSecurityDescriptor` attribute in the `top` class

NOTE

The lowercase `n` in the name `nTSecurityDescriptor` might look odd, but it's correct. While LDAP name lookups are case insensitive, the names themselves are defined using lower camel case.

To read the security descriptor, the user must be granted either `ReadControl` OR `AccessSystemSecurity` access rights on the object, depending on the parts of the security descriptor they've requested. [Listing 11-16](#) shows two techniques for retrieving the security descriptor of an Active Directory server object.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
❶ PS> $obj = Get-ADObject -Identity $root_dn -Properties "nTSecurityDescriptor"
PS> $obj.nTSecurityDescriptor.Access
ActiveDirectoryRights : ReadProperty
InheritanceType       : None
ObjectType             : 00000000-0000-0000-0000-000000000000
InheritedObjectType    : 00000000-0000-0000-0000-000000000000
ObjectFlags            : None
AccessControlType      : Allow
IdentityReference      : Everyone
IsInherited            : False
InheritanceFlags       : None
PropagationFlags       : None
--snip--

❷ PS> Format-Win32SecurityDescriptor -Name $root_dn -Type Ds
Path: DC=mineral,DC=local
Type: DirectoryService
Control: DaclPresent, DaclAutoInherited

<Owner>
- Name : BUILTIN\Administrators
- Sid  : S-1-5-32-544

<Group>
- Name : BUILTIN\Administrators
- Sid  : S-1-5-32-544

<DACL> (Auto Inherited)
- Type : AllowedObject
- Name : BUILTIN\Pre-Windows 2000 Compatible Access
- SID  : S-1-5-32-554
- Mask : 0x00000010
- Access: ReadProp
- Flags : ContainerInherit, InheritOnly
- ObjectType: 4c164200-20c0-11d0-a768-00aa006e0529
```



```
- InheritedObjectType: 4828cc14-1437-45bc-9b07-ad6f015e5f28
--snip--
```

Listing 11-16: Accessing the security descriptor for the root object

The first technique queries the object's security descriptor using `nTSecurityDescriptor` ❶. The `Get-ADObject` command automatically converts the security descriptor to an instance of the `.NET ActiveDirectorySecurity` class, so we can show its DACL using the `Access` property.

The second technique uses the Win32 security descriptor commands from the `NtObjectManager` module, specifying the `DS` type and the pathname as the distinguished name of the object. In this example, we use the `Format-Win32SecurityDescriptor` command ❷ to get the security descriptor and immediately format it.

When might you choose to use one technique over the other? The Win32 security descriptor commands are a better option if you have the `NtObjectManager` module installed, as they don't modify the information retrieved from the security descriptor. For example, you might notice that the first ACE in the DACL returned from each command isn't the same. One belongs to the *Everyone* user, whereas the other belongs to *BUILTIN\Pre-Windows 2000 Compatible Access*.

The difference comes from the fact that the `ActiveDirectorySecurity` class, which the `Get-ADObject` command uses to return the security descriptor from its attribute, automatically canonicalizes the DACL before allowing the user access to it. The canonicalization process might hide security misconfigurations. The Win32 command doesn't do any canonicalization.

Note that if you access the domain controller via the SAM remote service, you'll really be accessing the Active Directory server's user configuration, not a local SAM database. But if you inspect the security descriptors for the various supported objects, the SAM remote service won't return the Active Directory ones. Instead, the LSA will pick a security descriptor from a predefined set, choosing the one that most closely matches the one in the directory object. This is just for show, though; ultimately, any access checks will occur against the security descriptor stored in the Active Directory server.

Assigning Security Descriptors to New Directory Objects

When we create a new Active Directory object, we can assign it a security descriptor by providing a byte array for the object's `nTSecurityDescriptor` attribute. [Listing 11-17](#) shows how to set this security descriptor when running PowerShell as a domain administrator. Don't run these commands in a production environment, where modifying Active Directory could have adverse effects.

```
❶ PS> $sd = New-NtSecurityDescriptor -Type DirectoryService
PS> Add-NtSecurityDescriptorAce $sd -KnownSid BuiltinAdministrators
    -Access All
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
❷ PS> $obj = New-ADObject -Type "container" -Name "SDDEMO" -Path $root_dn
    -OtherAttributes @{nTSecurityDescriptor=$sd.ToByteArray()} -PassThru
PS> Format-Win32SecurityDescriptor -Name $obj.DistinguishedName -Type Ds
Path: cn=SDDEMO,DC=mineral,DC=local
Type: DirectoryService
Control: DaclPresent, DaclAutoInherited

<Owner>
  - Name   : MINERAL\Domain Admins
  - Sid    : S-1-5-21-146569114-2614008856-3334332795-512

<Group>
  - Name   : MINERAL\Domain Admins
  - Sid    : S-1-5-21-146569114-2614008856-3334332795-512

<DACL> (Auto Inherited)
❸ - Type   : Allowed
  - Name   : BUILTIN\Administrators
  - SID    : S-1-5-32-544
  - Mask   : 0x000F01FF
  - Access : Full Access
  - Flags  : None

❹ - Type   : AllowedObject
  - Name   : BUILTIN\Pre-Windows 2000 Compatible Access
  - SID    : S-1-5-32-554
  - Mask   : 0x00000010
  - Access : ReadProp
  - Flags  : ContainerInherit, InheritOnly, Inherited
  - ObjectType: 4c164200-20c0-11d0-a768-00aa006e0529
```

```
- InheritedObjectType: 4828cc14-1437-45bc-9b07-ad6f015e5f28
--snip--
```

Listing 11-17: Creating a new Active Directory object with a security descriptor

We first create a security descriptor containing a single ACE that grants the *Administrators* group full access ❶. We then create a new container object called *SDDEMO* using the `New-ADObject` command ❷, specifying the security descriptor using the `OtherAttributes` parameter.

Next, we format the new object's security descriptor. As you can see, the ACE we specified is at the top of the DACL ❸, but other ACEs have appeared after the one we specified ❹, as auto-inheritance rules apply to the DACL and SACL of the parent object. (As discussed in [Chapter 6](#), you can specify the `DaclProtected` and `SaclProtected` security descriptor control flags to prevent inheritable ACEs from being applied to the object, but we haven't done that here.)

What if we don't specify the security descriptor value when creating the object? In that case, the object will use a default security descriptor, taken from the schema class object's `defaultSecurityDescriptor` attribute. [Listing 11-18](#) shows how to manually create a new object security descriptor based on this default security descriptor attribute. This is simulating the operations the Active Directory server performs.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
❶ PS> $cls = Get-DsSchemaClass -Name "container"
❷ PS> $parent = Get-Win32SecurityDescriptor $root_dn -Type Ds
❸ PS> $sd = New-NtSecurityDescriptor -Parent $parent -EffectiveToken
-ObjectType $cls.SchemaId -Creator $cls.DefaultSecurityDescriptor
-Type DirectoryService -AutoInherit DaclAutoInherit, SaclAutoInherit
-Container
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : MINERAL\alice
<Group> : MINERAL\Domain Users
<DACL> (Auto Inherited)
MINERAL\Domain Admins: (Allowed)(None)(Full Access)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
--snip--

❹ PS> $std_sd = Edit-NtSecurityDescriptor $sd -Standardize -PassThru
```

```
❶ PS> Compare-NtSecurityDescriptor $std_sd $sd -Report
WARNING: DACL ACE 1 mismatch.
WARNING: Left : Type Allowed - Flags None - Mask 00020094 - Sid S-1-5-11
WARNING: Right: Type Allowed - Flags None - Mask 000F01FF - Sid S-1-5-18
WARNING: DACL ACE 2 mismatch.
WARNING: Left : Type Allowed - Flags None - Mask 000F01FF - Sid S-1-5-18
WARNING: Right: Type Allowed - Flags None - Mask 00020094 - Sid S-1-5-11
False
```

Listing 11-18: Creating a new object security descriptor

First, we get the `container` schema class ❶. By inspecting this class's schema identifier, we can determine which object ACEs were inherited (those with an `InheritedObjectType` value set) and identify the default security descriptors for the class. We then get the security descriptor from the parent, which is the root domain object ❷.

Next, we call `New-NtSecurityDescriptor`, specifying the parent security descriptor, the default security descriptor, and the object type ❸. We also specify the auto-inherit flags, to automatically inherit any DACL or SACL ACEs, and use the `Container` parameter to identify that the security descriptor will secure a container, which ensures that it will use the correct inheritance rules. Finally, we can format the newly created security descriptor, which has auto-inherited the DACL.

The new security descriptor has the owner and group SIDs you might expect: namely, the user SID and the primary group SID of the `Token` object on which it is based. However, this won't always be the case. If the creator of the object is a local administrator on the Active Directory server, the server will change the owner and group SIDs to one of the following SIDs:

Domain Admins Set for any object in the default naming context under the domain root

Enterprise Admins Set for any object in the configuration naming context

Schema Admins Set for any object in the schema naming context

Changing the owner and group SIDs to one of these values ensures that the resources across a forest have appropriate owners. For example, if *Enterprise Admins* weren't the default owner for configuration objects, an administrator from a different domain in the forest might create an object that an administrator in another domain wouldn't be able to access, even if they were in the correct group.

To create the final security descriptor, we must perform one last step: standardization. *Security descriptor standardization* is a feature introduced in Windows Server 2003, and it's turned on by default. It ensures that non-inherited ACEs always appear in a binary comparison order. This contrasts with the ACL canonicalization process described in [Chapter 5](#), which orders the ACEs based on the ACE type rather than on their binary value. Consequently, two canonical ACLs with the same ACE entries could have different ordering.

We can standardize a security descriptor using the `Edit-NtSecurityDescriptor` command and the `Standardize` parameter ④. Note, however, that the standardized ACL form doesn't always match the canonical one. Indeed, if we compare the original canonicalized security descriptor (shown in [Listing 11-16](#)) with the standardized one, the `Compare-NtSecurityDescriptor` command shows two reordered ACEs ⑤. In theory this discrepancy could change the result of an access check, but in practice it's unlikely to do so, as `Denied` ACEs always appear before `Allowed` ACEs, regardless of the other ACE ordering rules in place.

An administrator can disable the standardization feature by setting a flag in the directory's special `dsHeuristics` attribute. You can query this flag using the `Get-DsHeuristics` PowerShell command, as shown in [Listing 11-19](#).

```
PS> (Get-DsHeuristics).DontStandardizeSDs
False
```

Listing 11-19: Checking whether security descriptor standardization is enabled

If the command returns `True`, security descriptor standardization is disabled.

Assigning Security Descriptors to Existing Objects

You can use the `Set-Win32SecurityDescriptor` PowerShell command to change an existing object's security descriptor based on the distinguished name of the object. [Listing 11-20](#) demonstrates this for the object `CN=SomeObject,DC=mineral,DC=local`. Before running the script, change this name to that of an object that exists in your Active Directory configuration.

```
PS> $dn = "CN=SomeObject,DC=mineral,DC=local"
PS> $sd = New-NtSecurityDescriptor "D:(A;;GA;;;WD)"
PS> Set-Win32SecurityDescriptor $dn -Type Ds -SecurityDescriptor $sd -SecurityInformation Dacl
```

Listing 11-20: Setting an object's security descriptor using the `Set-Win32SecurityDescriptor` command

The command sends a modification request to the directory server to set the `NtSecurityDescriptor` attribute. As discussed in [Chapter 6](#), the user modifying the security descriptor must be granted the appropriate access rights on the object (such as `WriteDac` access) for the part of the security descriptor being written.

Security information flags specify which parts of the security descriptor you can modify. To get this information, request the constructed `sDRightsEffective` attribute for the object. The `Get-DsSDRightsEffective` PowerShell command exposes this attribute, as shown in [Listing 11-21](#).

```
PS> Get-DsSDRightsEffective -DistinguishedName $dn
Owner, Group, Dacl
```

Listing 11-21: Querying for the effective security information

The output indicates that the current caller would be granted write access to the owner, group, and DACL. This result takes into account privileges such as `SeTakeOwnershipPrivilege`, which allows a caller to modify the owner even if the security descriptor doesn't grant `WriteOwner` access. The directory also allows a caller to bypass certain checks using privileges; for ex-

ample, it can check for `SeRestorePrivilege` to determine whether the caller can set arbitrary owner SIDs.

NOTE

To add or remove a DACL-protected flag with the `Set-Win32SecurityDescriptor` command, you'll need to use the `ProtectedDacl` or `UnprotectedDacl` security information flag. These flags aren't passed to the server; instead, they are set in the security descriptor's control flags, which are then sent to the server.

In [Listing 11-22](#), we build a new security descriptor for an object, deriving it from three values: the security descriptor supplied by the user, the current security descriptor, and the parent security descriptor.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $user_dn = "CN=Users,$root_dn"
❶ PS> $curr_sd = Get-Win32SecurityDescriptor "CN=Users,$root_dn" -Type Ds
PS> Format-NtSecurityDescriptor $curr_sd -Summary
<Owner> : DOMAIN\Domain Admins
<Group> : DOMAIN\Domain Admins
<DACL> (Auto Inherited)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
--snip--

❷ PS> $new_sd = New-NtSecurityDescriptor "D:(A;;GA;;;WD)"
❸ PS> Edit-NtSecurityDescriptor -SecurityDescriptor $curr_sd
-NewSecurityDescriptor $new_sd -SecurityInformation Dacl
-Flags DaclAutoInherit, SaclAutoInherit

PS> $cls = Get-DsObjectSchemaClass $user_dn
PS> $parent = Get-Win32SecurityDescriptor $root_dn -Type Ds
❹ PS> $sd = New-NtSecurityDescriptor -Parent $parent
-ObjectType $cls.SchemaId -Creator $curr_sd -Container
-Type DirectoryService -AutoInherit DaclAutoInherit, SaclAutoInherit,
AvoidOwnerCheck, AvoidOwnerRestriction, AvoidPrivilegeCheck
-EffectiveToken

❺ PS> Edit-NtSecurityDescriptor $sd -Standardize
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : DOMAIN\Domain Admins
<Group> : DOMAIN\Domain Admins
<DACL> (Auto Inherited)
```



```
Everyone: (Allowed)(None)(Full Access)
--snip--
```

Listing 11-22: Creating a new security descriptor for an object

First, we get the current security descriptor for the object. In this case I've picked the `Users` container, as it provides an easy example ❶, but you can choose any object in the directory. Next, we create a new security descriptor ❷ and use the `Edit-NtSecurityDescriptor` PowerShell command to modify the object's existing security descriptor, replacing it with the one we just created ❸. In this command, we must specify the security information flags as well as the auto-inherit flags.

We then use the modified security descriptor as the creator security descriptor, using the parent security descriptor and the target object's class information for inheritance ❹. We specify some additional auto-inherit flags to disable the owner check; this ensures that we set the owner value correctly based on the original security descriptor. Disabling the checks isn't a security issue because the caller must have set the `Owner` security information flag to change the owner, and `Edit-NtSecurityDescriptor` would have checked for the owner SID, preventing a user from circumventing the check.

We can now standardize the security descriptor and format it ❺. As you can see, it now contains the `Everyone` ACE, matching the new security descriptor we specified. At this point, the server will also enumerate any child objects of the security descriptor we're modifying and apply any inheritance changes to the new security descriptor we've introduced.

Note that the server automatically propagates inheritable ACEs to child objects whenever a parent object's security descriptor changes. This behavior contrasts with that of files and registry keys, where it's the responsibility of the Win32 APIs to manually propagate inheritance to children. The automatic propagation introduces an interesting consequence: the server doesn't check that the user setting the security descriptor has appropriate access rights to the child object. Therefore, a user with `WriteDac` access to an object higher in a hierarchy can set a new inheritable ACE and grant themselves access to a child object to which they didn't previously have access.

The only way to mitigate this behavior is by setting the `DaclProtected` control flag in the object's security descriptor to block inheritance (as well as the fact that administrators should never grant `WriteDac` access to non-administrator users).

Inspecting a Security Descriptor's Inherited Security

Because the security descriptors are assigned based on the object hierarchy, it's possible to locate the source of their inherited ACEs using the `Search-Win32SecurityDescriptor` PowerShell command. In [Listing 11-23](#), we find the inherited ACEs for the `Users` container.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $user_dn = "CN=Users,$root_dn"
PS> $cls = Get-DsObjectSchemaClass -DistinguishedName $user_dn
PS> Search-Win32SecurityDescriptor -Name $user_dn -Type Ds
-ObjectType $cls.SchemaId
```

Name	Depth	User	Access
----	-----	----	-----
	0	NT AUTHORITY\SYSTEM	GenericAll
	0	MINERAL\Domain Admins	CreateChild ...
	0	BUILTIN\Account Operators	CreateChild ...
	0	BUILTIN\Account Operators	CreateChild ...
	0	BUILTIN\Print Operators	CreateChild ...
	0	NT AUTHORITY\Authenticated Users	GenericRead
	0	BUILTIN\Account Operators	CreateChild ...
DC=mineral,DC=local	1	BUILTIN\Pre-Windows 2000...	ReadProp
DC=mineral,DC=local	1	BUILTIN\Pre-Windows 2000...	ReadProp
DC=mineral,DC=local	1	BUILTIN\Pre-Windows 2000...	ReadProp

Listing 11-23: Searching for the source of inherited ACEs

You can use this command with Active Directory objects in almost the same way as you would use it with files. The important difference is that you must set the `Type` property to `Ds` to look up Active Directory objects on the server.

You must also specify the schema class GUID for inheritance ACEs using the `ObjectType` parameter; otherwise, the command might not be able to find the source ACEs at all, as they're likely to be inherited based on the object's type. In my testing, the search sometimes succeeded when I didn't

specify the object type, but in most cases, the operation failed with an unrelated error.

Access Checks

Now that we can query an object's security descriptor, we can perform an access check to determine whether it would grant a user some specific access. Active Directory designates nine type-specific access rights that directory objects can grant, in addition to the standard rights such as `ReadControl` and `WriteDac` (used to read and write, respectively, the security descriptor on the object). They are:

`CreateChild` Enables creating a new child object

`DeleteChild` Enables deleting a child object

`List` Enables listing child objects

`Self` Enables writing an attribute value (which the server will verify)

`ReadProp` Enables reading an attribute value

`WriteProp` Enables writing an attribute value

`DeleteTree` Enables deleting a tree of objects

`ListObject` Enables listing a specific object

`ControlAccess` Grants access to a directory operation

Some of these access rights require more explanation than others. In the following sections, we'll walk through the various operations they represent and how they're used to determine what a user can do on the directory server. Note that the behaviors of these access rights also apply to ACEs specified in an object's SACL, meaning you should be able to take the descriptions presented here and apply them to the generation of audit events.

Creating Objects

If a user is granted the `CreateChild` access right, they can create a child object for the object. The object's `AllowedObject` ACEs determine what kinds of child objects a user can create. [Listing 11-24](#) shows how to grant the `CreateChild` access right for a specific object type.

```
PS> $sd = New-NtSecurityDescriptor -Type DirectoryService -Owner "SY"
-Group "SY"
❶ PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type Allowed
-Access List
❷ PS> $user = Get-DsSchemaClass -Name "user"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
-Access CreateChild -ObjectType $user.SchemaId
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl
-ResolveObjectType
<Dacl>
Everyone: (Allowed)(None)(List)
Everyone: (AllowedObject)(None)(CreateChild)(OBJ:User)

❸ PS> Get-NtGrantedAccess $sd -ObjectType $user
CreateChild, List

❹ PS> $cont = Get-DsSchemaClass -Name "container"
PS> Get-NtGrantedAccess $sd -ObjectType $cont
List
```

Listing 11-24: Testing CreateChild object type access

We first create a new security descriptor and add an ACE that grants everyone `List` access ❶. This ACE doesn't specify an object type, so it will apply to every user who matches the SID. Next, we get the `user` schema class ❷ and use it to create a second ACE that grants `CreateChild` access, specifying the schema identifier as the object type.

We display the security descriptor to verify that we've created the correct ACEs, passing the `ResolveObjectType` parameter to `Format-NtSecurityDescriptor` to return the directory object type's name. If you don't use this parameter, the command will print the GUID instead, which is less useful; however, note that returning these names can be quite time-consuming and might cause the command to hang.

We now request the maximum granted access for the security descriptor ❸, specifying the schema class as the object type to check, and are granted `CreateChild` and `List` access. The directory server will do the same when performing the access check for the child creation operation; it will look up the schema class identifier for the object class being created and pass it to the access check API. If `CreateChild` access is granted, the operation will proceed.

Finally, we repeat the access check but instead specify the `container` class ❹. This time, we're granted only `List` access—because we didn't pass the `user` class's identifier in the list of object types to check, the access check ignored the `CreateChild` ACE.

If an object's security descriptor contains an ACE that grants the `CreateChild` access right with no object type specified, the user can create any child object. However, limitations still exist. First, the user can only create new objects of structural classes; the server should reject the creation of an object from an abstract or auxiliary class. Second, each schema class has a list of possible parent classes, or *superiors*, stored in the `possSuperiors` and `systemPossSuperiors` attributes. The server will permit the creation of a child only if the parent object's class is in this list of classes.

Determining all permitted child classes can be quite complex due to the rules of class inheritance. Fortunately, the directory server also constructs the `possibleInferiors` attribute, which lists the classes the directory will allow as children for a given schema class. You can query for these classes using the `Get-DsSchemaClass` PowerShell command with the `Inferior` parameter, as shown in [Listing 11-25](#).

```
PS> Get-DsSchemaClass "user" -Inferior
```

Name	SchemaId	Attributes
----	-----	-----
ms-net-ieee-80211-GroupPolicy	1cb81863-b822-4379-9ea2-5ff7bdc6386d	3
nTFRSSubscriptions	2a132587-9373-11d1-aebc-0000f80367c1	3
classStore	bf967a84-0de6-11d0-a285-00aa003049e2	4
ms-net-ieee-8023-GroupPolicy	99a03a6a-ab19-4446-9350-0cb878ed2d9b	3

Listing 11-25: Listing inferior classes of the user schema class

Listing 11-25 shows the four child classes allowed for a user object. Trying to create an object of a class that isn't in the list of children will result in an error and abort the creation operation. An administrator can change this list by adding the `user` class to another class's `possSuperiors` attribute.

ABUSING CHILD CLASSES

If a user is granted the `CreateChild` access right, there is a risk that they could configure the directory outside of the expected limits. You should assume that granting the ability to create a child means the user can set any attribute in the new object, some of which might inform security decisions made by the server or third-party applications. The user can also create new objects with inferior classes permitted.

When might the ability to create inferior classes lead to problems? As an example, I found a class added to the Active Directory server when the Exchange mail server was installed that normal users could create in existing objects in the directory. This class, in turn, had the `container` class as an inferior, which could contain security-critical classes such as `user` or `group`. Look up CVE-2021-34470 to read the details of this issue.

You can pipe the output of one `Get-DsSchemaClass` command to another to build the full list of child classes originating from a parent:

```
PS> Get-DsSchemaClass user -Inferior | Get-DsSchemaClass -Inferior
```

This will show what object types you could create if you had `CreateChild` access. Repeat the pipeline until you stop receiving new classes in the output.

Deleting Objects

Three access rights control deletion: `Delete`, `DeleteChild`, and `DeleteTree`. Each concerns a different delete operation. The `Delete` access right applies only to the current object; if the object has child objects, the server will refuse to delete the object. (A client application could bypass this restric-

tion by recursively enumerating all children and deleting them if the user had the necessary access.)

If the user is granted `DeleteChild` access, they can delete any immediate child object, although if that child object has its own children, the same restriction as for `Delete` applies. The ACE granting `DeleteChild` access can use the object type to restrict which of an object's classes a user can delete.

Finally, the `DeleteTree` access right allows a user to delete an entire tree of objects, including the root object. This deletion is performed entirely on the server, using a specific tree-deletion command. The user does not need any deletion rights on the child objects if they have this right.

You can remove objects using the `Remove-ADObject` PowerShell command. To use the `DeleteTree` access right, you must specify the `Recursive` parameter.

Listing Objects

The list of access rights includes two rights for listing objects, `List` and `ListObject`. There are some differences between these. By default, if a user is not granted `List` access, they cannot inspect any of an object's children. However, this restriction isn't transitive; for example, if a child object grants the `List` access right, the user can inspect the children of that object, even though they can't list the object itself from the parent. (This means the user will need to know the name of the child object to inspect.)

`ListObject` access applies not to the parent but to individual objects. If a user has the `ListObject` access right on an object but doesn't have the `List` access right on the parent, the user can still list and interact with the object. By default, the Active Directory server doesn't check the `ListObject` access right, likely for performance reasons.

If the user were not granted `List` access on an object, but tried to enumerate its children, the server would need to do an access check for every child object to find out which were visible through allowing `ListObject` access. For directory objects with large numbers of children, this would be a very expensive operation.

You can enable this access right using a flag in the `dsHeuristics` attribute in the directory. Query the flag using the `Get-DsHeuristics` PowerShell command:

```
PS> (Get-DsHeuristics).DoListObject
```

If the output is `True`, the `ListObject` access right is enabled.

Reading and Writing Attributes

The `ReadProp` and `WriteProp` access rights control the reading and writing, respectively, of attributes in an object. It's possible to allow the reading and writing of all of an object's attributes through an ACE with no object type. More commonly, however, an object will allow the reading of all attributes, but restrict which attributes can be written by specifying an ACE's object type as the attribute's schema identifier.

Listing 11-26 shows an example of how to implement an access check for reading and writing attributes.

```
❶ PS> $sd = New-NtSecurityDescriptor -Type DirectoryService -Owner "DA"  
-Group "DA"  
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type Allowed  
-Access ReadProp  
❷ PS> $attr = Get-DsSchemaAttribute -Name "accountExpires"  
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject  
-Access WriteProp -ObjectType $attr.SchemaId  
❸ PS> Get-NtGrantedAccess $sd -ObjectType $attr  
ReadProp, WriteProp  
❹ PS> $pwd = Get-DsSchemaAttribute -Name "pwdLastSet"  
PS> Get-NtGrantedAccess $sd -ObjectType $pwd  
ReadProp
```

Listing 11-26: Testing the ReadProp and WriteProp access rights

We start by creating a new security descriptor with an `Allowed` ACE that grants `ReadProp` access, without specifying an object type ❶. We then add an ACE that grants `WriteProp` access to only the `accountExpires` attribute ❷.

Next, we perform an access check specifying that attribute's schema identifier as the object type ❸, and we're granted both `ReadProp` and `WriteProp` access. However, if we run the access check with a different attribute type ❹, we're granted only the general `ReadProp` access.

Note that the security descriptor could contain a `Denied` ACE to block the reading or writing of a specific attribute, even if a separate ACE enabled reading or writing of all attributes. For instance, if the `Denied` ACE blocked the reading of the `pwdLastSet` attribute we queried for here, even `ReadProp` access wouldn't be granted. The directory server must ensure that it specifies the exact object type for the attributes to check.

NOTE

Even if the access check indicates that an attribute can be read or written, the directory server doesn't have to honor that decision. The directory contains several attributes that a normal user can't read or write. For example, they can't read or write user passwords, which are stored in the `unicodePwd` attribute that only the system is permitted to access. No amount of configuring the security descriptor should change this behavior (although a separate mechanism allows a user to write the password; we'll come back to this in "Control Access Rights" on page 376). Note also that a normal user can't modify any attribute that is marked as system-only, indicated by the `systemOnly` attribute in the schema.

Checking Multiple Attributes

To avoid making you send multiple requests to the directory server, LDAP supports the reading and writing of multiple attributes in a single request. However, it would be expensive to then require an access check for each of these attributes' schema identifiers before determining what you can read or write.

As I described in [Chapter 7](#), the access check process allows you to build a tree of object types to verify multiple attributes in a single check. This tree lists each object type and what access it will be granted, enabling the directory server to quickly determine if it should grant a request. [Listing](#)

11-27 shows how to use an object type tree in an access check. It adds to the commands in **Listing 11-26**.

```
❶ PS> $user = Get-DsSchemaClass -Name "user"
PS> $obj_tree = New-ObjectTypeTree $user
PS> Add-ObjectTypeTree -Tree $obj_tree $attr
PS> Add-ObjectTypeTree -Tree $obj_tree $pwd

❷ PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
Format-Table Status, SpecificGrantedAccess, Name
      Status SpecificGrantedAccess Name
      -----
STATUS_SUCCESS          ReadProp user
STATUS_SUCCESS ReadProp, WriteProp accountExpires
STATUS_SUCCESS          ReadProp pwdLastSet

❸ PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult
-Access WriteProp | Format-Table Status, SpecificGrantedAccess, Name
      Status SpecificGrantedAccess Name
      -----
STATUS_ACCESS_DENIED          None user
      STATUS_SUCCESS          WriteProp accountExpires
STATUS_ACCESS_DENIED          None pwdLastSet
```

Listing 11-27: Using an object type tree to check multiple attributes

We first get the user schema class ❶ and use it to build the tree, setting the class's schema identifier as the tree's root. We then add the two attributes we want to check, accountExpires and pwdLastSet, as leaf nodes to the root, using the Add-ObjectTypeTree command. **Figure 11-4** shows the structure of the final tree.

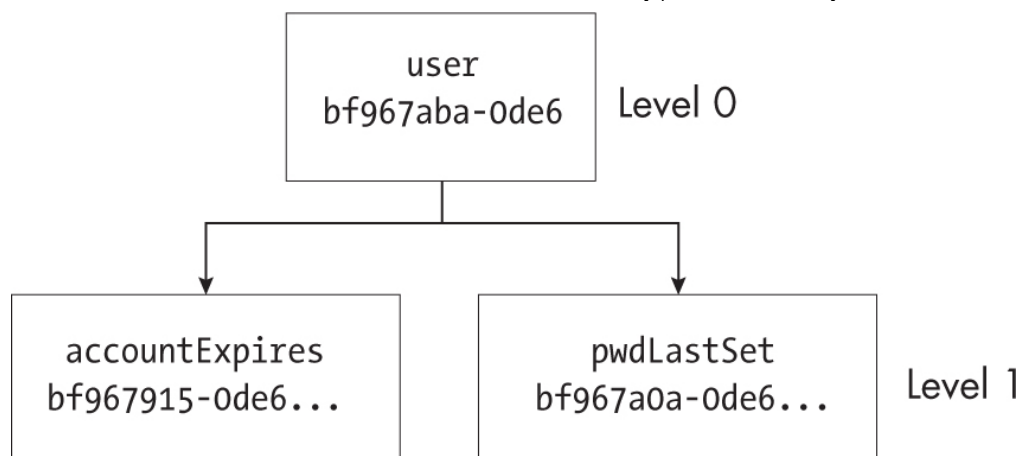


Figure 11-4: The object type tree for the user object and its `accountExpires` and `pwdLastSet` attributes

Next, we pass the tree to `Get-NtGrantedAccess` ❷, making sure to specify that we want the list of all results, not the single granted-access value. The results show that only the `accountExpires` attribute has been granted `ReadProp` and `WriteProp` access, while the `user` object and `pwdLastSet` attribute have been granted `ReadProp` access only.

Typically, the Active Directory server will specify an explicit access right to check for, rather than simply requesting the maximum granted access. We can test this by specifying the `Access` parameter with a value of `WriteProp` and checking the resulting behavior ❸. The results show that the `user` object and its `pwdLastSet` attribute have been denied access, but that the `accountExpires` attribute is granted `WriteProp` access.

The fact that the object's class is specified in the tree leads to an interesting behavior, demonstrated in [Listing 11-28](#).

```

PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access WriteProp -ObjectType $user.SchemaId
PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
Format-Table Status, SpecificGrantedAccess, Name
      Status SpecificGrantedAccess Name
      -----
STATUS_SUCCESS ReadProp, WriteProp user
STATUS_SUCCESS ReadProp, WriteProp accountExpires
STATUS_SUCCESS ReadProp, WriteProp pwdLastSet
  
```

Listing 11-28: Granting `WriteProp` access to the schema class

As you can see, it's possible to add an ACE that grants access rights for all attributes of a specified object class. Here, we add an ACE granting `WriteProp` access and specify the `user` class's schema identifier. When we repeat our access check, this time we find that `WriteProp` access is granted for all attributes in the tree.

This behavior, granting access to all attributes, is likely an emergent property of the implementation, not an intentional design decision; the Windows user interface for modifying a directory object's security descriptor can't understand the ACE and shows it as granting no specific access rights. An attacker could use this behavior to hide malicious modifications to the security descriptor from an administrator.

Analyzing Property Sets

As shown in [Listing 11-29](#), an object class can have many attributes—in the case of the `user` class, a total of 428 if we include the attributes of all its auxiliary classes.

```
PS> (Get-DsSchemaClass user -Recurse -IncludeAuxiliary |  
Sort-Object SchemaId -Unique |  
Select-Object -ExpandProperty Attributes).Count  
428
```

Listing 11-29: Counting attributes for the user schema class

If you wanted to grant specific access rights to all of these attributes, the DACL would quickly become unmanageable; the ACL might even run out of its allowed 64KB of space.

To partially solve this problem, the Active Directory configuration can define arbitrary *property sets*, which group multiple attributes together under a single GUID. It can then use this identifier as the object type in an ACE to grant or deny access to a group of attributes in one go. Property sets are just one type of *extended right*, which allow an administrator to add additional access rights to the directory. We'll cover the other two, control access rights and validated write access rights, in the following sections. [Listing 11-30](#) shows how to get all the extended rights in the current directory.

```

PS> $config_dn = (Get-ADRootDSE).configurationNamingContext
PS> $extended_dn = "CN=Extended-Rights,$config_dn"
PS> Get-ADObject -SearchBase $extended_dn -SearchScope OneLevel -Filter *
-Properties * | Group-Object {
    Get-NtAccessMask $_.validAccesses -AsSpecificAccess DirectoryService
}
Count Name                                Group
-----
60 ControlAccess                          {CN=Add-GUID,CN=Extended-Rights,...}
15 ReadProp, WriteProp                     {CN=DNS-Host-Name-Attributes,...}
6 Self                                    {CN=DS-Validated-Write-Computer,...}

```

Listing 11-30: Getting extended rights and grouping them by the validAccesses attribute

An object can specify a particular type of extended right in its `validAccesses` attribute, which stores an integer representing directory object access rights. We convert the attribute to an access rights enumeration using the `Get-NtAccessMask` PowerShell command. If the `validAccesses` attribute (and thus the value in the `Name` column) is set to `ReadProp` and `WriteProp`, the extended right is a property set.

To simplify the analysis of extended rights and property sets, the `NtObjectManager` module implements the `Get-DsExtendedRight` PowerShell command, as shown in [Listing 11-31](#).

```

❶ PS> $attr = Get-DsSchemaAttribute -Name "accountExpires"
PS> $prop_set = Get-DsExtendedRight -Attribute $attr
PS> $prop_set
Name                                RightsId
----
❷ User-Account-Restrictions 4c164200-20c0-11d0-a768-00aa006e0529

❸ PS> $prop_set.AppliesTo | Select-Object Name
Name
----
msDS-GroupManagedServiceAccount
inetOrgPerson
msDS-ManagedServiceAccount
computer
user

```

```

❶ PS> $user = Get-DsSchemaClass user
PS> Get-DsExtendedRight -SchemaClass $user
Name                                RightsId
----                                -
Allowed-To-Authenticate             68b1d179-0d15-4d4f-ab71-46152e79a7bc
Email-Information                   e45795b2-9455-11d1-aebd-0000f80367c1
General-Information                 59ba2f42-79a2-11d0-9020-00c04fc2d3cf
--snip--

```

Listing 11-31: Getting the property set for an attribute and its possible schema classes

We first get the `accountExpires` attribute we used earlier and pass it to the `Get-DsExtendedRight` command ❶. If the attribute is part of a property set, the command will return the extended right. Here, the output lists the attribute as part of the `User-Account-Restrictions` property set ❷.

The `RightsId` column provides the GUID you'd use in an ACE to allow or deny access to the object type. You can find this GUID in the schema attribute's `attributeSecurityGUID` attribute. Each property set also has a list of schema classes that are allowed to contain it ❸. This allows the directory server to know what object type tree it needs to build when doing an access check.

Finally, we perform the reverse operation; finding all property sets that apply to a specific schema class, `user` ❹.

Listing 11-32 demonstrates using a property set in an access check.

```

❶ PS> $sd = New-NtSecurityDescriptor -Type DirectoryService
    -Owner "SY" -Group "SY"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access ReadProp -ObjectType $prop_set.RightsId
❷ PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access WriteProp -ObjectType $attr.SchemaId
❸ PS> $obj_tree = New-ObjectTypeTree -SchemaObject $user
PS> Add-ObjectTypeTree -Tree $obj_tree -SchemaObject $prop_set
❹ PS> Get-NtGrantedAccess $sd -ObjectType $prop_set -ResultList -PassResult |
    Format-Table SpecificGrantedAccess, Name
SpecificGrantedAccess Name
-----

```

```

ReadProp user
ReadProp User-Account-Restrictions
ReadProp, WriteProp accountExpires
ReadProp msDS-AllowedToActOnBehalfOfOtherIdentity
ReadProp msDS-User-Account-Control-Computed
ReadProp msDS-UserPasswordExpiryTimeComputed
ReadProp pwdLastSet
ReadProp userAccountControl
ReadProp userParameters

```

Listing 11-32: Performing an access check with a property set

We build a new security descriptor to do the check ❶, and we grant `ReadProp` access based on the property set identifier. We also grant `WriteProp` access to the `accountExpires` attribute within that set, using the `attr` variable we defined in [Listing 11-31](#) ❷.

Next, we need to build the object type tree ❸. As before, the root of the tree is the object class. We then add the property set as a child of the tree, producing the object type tree shown in [Figure 11-5](#).

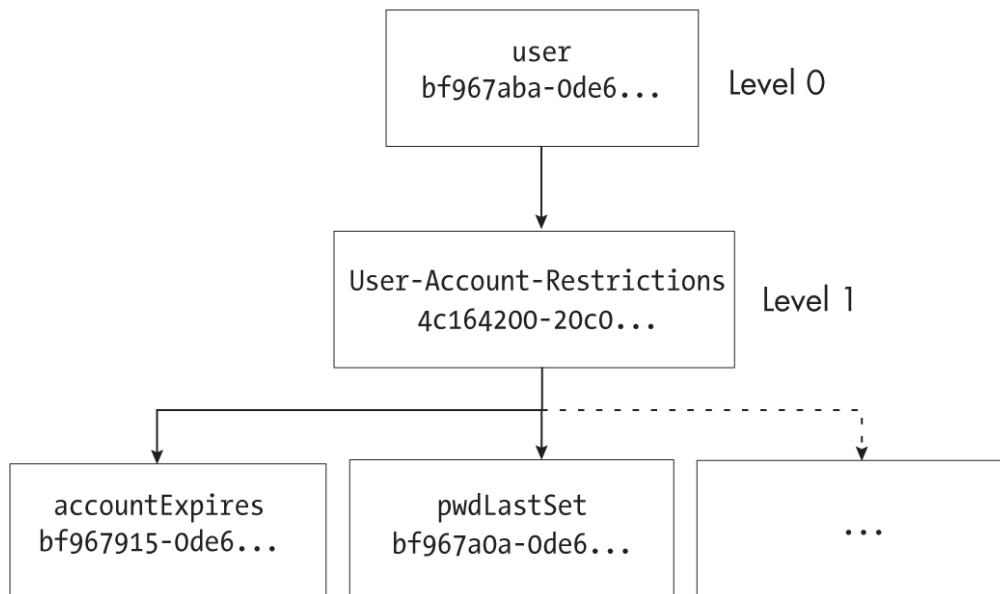


Figure 11-5: The property set object type tree

This object type tree contains both the property set at level 1 and entries for each attribute in the set at level 2. This tree structure allows us to grant access based on either the property set identifier or individual attributes.

Note that the directory server implements individual attribute checks a little differently; it always uses property sets if it can, but if an attribute isn't in a property set it uses a dummy GUID, named `PROPSET_GUID_DEFAULT`, as a placeholder. You might see this GUID in audit log entries, although the configuration's extended rights don't specify it.

We pass the object type tree and security descriptor to the access check ❹, and since we granted the property set `ReadProp` access, all attributes in the set receive at least this level of access. Because we explicitly granted `WriteProp` access to the `accountExpires` attribute, it receives this access right as well.

As you can see, if the security descriptor granted `WriteProp` access to every attribute in the set, the access would propagate to the property set node at level 1. Therefore, if the server merely checked the property set's granted access, it wouldn't matter if the security descriptor granted the access directly, using the property set's identifier, or instead granted access to every individual attribute in the set.

One last thing to highlight is what happens when we add a `Denied` ACE for attributes in a property set. [Listing 11-33](#) shows an example.

```
❶ PS> $pwd = Get-DsSchemaAttribute -Name "pwdLastSet"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type DeniedObject
    -Access ReadProp -ObjectType $pwd.SchemaId
❷ PS> Edit-NtSecurityDescriptor $sd -CanonicalizeDACL
PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
Format-Table SpecificGrantedAccess, Name
SpecificGrantedAccess Name
-----
❸ None user
    None User-Account-Restrictions
ReadProp, WriteProp accountExpires
ReadProp msDS-AllowedToActOnBehalfOfOtherIdentity
ReadProp msDS-User-Account-Control-Computed
ReadProp msDS-UserPasswordExpiryTimeComputed
    None pwdLastSet
ReadProp userAccountControl
ReadProp userParameters
```

Listing 11-33: Denying access to an attribute in a property set

In this listing, we include a Denied ACE for the `pwdLastSet` attribute to restrict the `ReadProp` access right ❶. You must remember to canonicalize the DACL ❷ after adding the ACE; otherwise, it won't appear at the start of the list, and the access check process will ignore it.

When we run the access check, we can see that the Denied ACE has removed `ReadProp` access from the `pwdLastSet` attribute, then propagated that change to the property set and user class, removing their access as well ❸. All other attributes in the set retain their `ReadProp` access. This behavior makes sense: if one of the property set's attributes is denied access, then the property set as a whole isn't granted `ReadProp` access.

If the property set identifier was used for the DeniedObject ACE, all attributes in the set would be denied the `ReadProp` access right. However, `accountExpires` would still be granted `WriteProp` access as it has a separate ACE granting it that access.

An Active Directory server administrator can add their own property sets to the configuration to extend this functionality to commonly used attributes; this reduces the complexity of object security descriptors.

Inspecting Control Access Rights

The second type of extended right, *control access rights*, don't necessarily correspond to any object attribute; instead, they tell the Active Directory server whether the user can perform a particular operation. Let's start by listing a subset of the control access rights, as shown in [Listing 11-34](#).

```
PS> Get-DsExtendedRight | Where-Object {
    $_.IsControl -and $_.Name -match "password"
} | Select-Object Name, RightsId
```

Name	RightsId
----	-----
User-Force-Change-Password	00299570-246d-11d0-a768-00aa006e0529
Unexpire-Password	ccc2dc7d-a6ad-4a7a-8846-c04e3cc53501
Update-Password-Not-Required-Bit	280f369c-67c7-438e-ae98-1d46f3c6f541
User-Change-Password	ab721a53-1e2f-11d0-9819-00aa0040529b

Listing 11-34: Listing control access rights with password in the name

Using the `IsControl` property, we filter the output so it includes only control access rights with `password` in their name. The `IsControl` property is `true` if the `validAccesses` attribute on the extended right is set to `ControlAccess`. The results include two commonly used control access rights, `User-Change-Password` and `User-Force-Change-Password`, which allow a user to modify their user object's `unicodePwd` write-only attribute. We can't grant this ability using `WriteProp` `ACCESS`.

The difference between these two rights is that `User-Change-Password` requires the user to send their old password as part of the modify operation, while `User-Force-Change-Password` works without requiring the old password. These correspond to the `ChangePassword` and `ForcePasswordChange` SAM user access rights we discussed in [Chapter 10](#) and serve the same purpose.

To give an example of how the directory server might check for a control access right, let's assume a user wants to change another user's password. [Listing 11-35](#) shows how the server might implement the access check for permitting the change operation.

```
❶ PS> $sd = New-NtSecurityDescriptor -Type DirectoryService -Owner "SY"
   -Group "SY"
PS> $right = Get-DsExtendedRight -Name 'User-Change-Password'
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
   -Access ControlAccess -ObjectType $right.RightsId
❷ PS> $user = Get-DsSchemaClass user
PS> $obj_tree = New-ObjectTypeTree -SchemaObject $user
PS> Add-ObjectTypeTree -Tree $obj_tree -SchemaObject $right
❸ PS> $force = Get-DsExtendedRight -Name 'User-Force-Change-Password'
PS> Add-ObjectTypeTree -Tree $obj_tree -SchemaObject $force
❹ PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
   Format-Table Status, SpecificGrantedAccess, Name
               Status SpecificGrantedAccess Name
               -----
STATUS_ACCESS_DENIED          None user
STATUS_SUCCESS                ControlAccess User-Change-Password
STATUS_ACCESS_DENIED          None User-Force-Change-Password
```

Listing 11-35: Checking for the User-Change-Password control access right

First, we create a new security descriptor, get the control access right, and add an ACE to the security descriptor granting the `ControlAccess` access right for `User-Change-Password` ❶. Next, we query for the `user` schema class and use it to build the object type tree ❷. We need the object class to be the root, but we make the control access right its immediate child. We also query for the `User-Force-Change-Password` control access right and add it to the tree ❸. If the user is granted this right, the server will allow them to force the password change even if they cannot provide the currently set password.

We then run the access check ❹ and see that the user has been granted `ControlAccess` for the `User-Change-Password` control access right. Now the directory server can proceed with the operation.

As with other types of access, it's possible for a security descriptor to grant `ControlAccess` either with a non-object ACE or on the object class. From the access check perspective, `ControlAccess` is granted to the control access right; the directory server doesn't necessarily know the difference. It's also possible for an administrator to extend the list of control access rights, although that normally requires a third-party application to check for the right, as the directory server won't know about it.

Analyzing Write-Validated Access Rights

The final type of extended right is write-validated access rights. They're defined when the `validAccesses` attribute is set to `Self`. **Listing 11-36** shows how to list the write-validated access rights by filtering on the `IsValidatedWrite` property.

```
PS> Get-DsExtendedRight | Where-Object IsValidatedWrite
```

Name	RightsId
Validated-MS-DS-Behavior-Version	d31a8757-2447-4545-8081-3bb610cacbf2
Self-Membership	bf9679c0-0de6-11d0-a285-00aa003049e2
Validated-MS-DS-Additional-DNS-Host-Name	80863791-dbe9-4eb8-837e-7f0ab55d9ac7
Validated-SPN	f3a64788-5306-11d1-a9c5-0000f80367c1
DS-Validated-Write-Computer	9b026da6-0d3c-465c-8bee-5199d7165cba

Listing 11-36: Listing write-validated access rights

A write-validated access right grants a user the ability to write to certain attributes of an object, with the server verifying the new value for the attribute before it's written. As an example, if a user wants to add a new member to a group object, they will need `WriteProp` access on the `member` attribute, which contains a list of distinguished names of all users and groups that are members of that group. Being granted `WriteProp` access will allow the user to modify the member list, adding or removing user or group objects. A user without that access right might still be able to add or remove their own user account name, however, if they're granted the `Self` access right for the `Self-Membership` write-validated access right on a group object. While this operation would still modify the `member` attribute, the server would ensure that the added or removed value corresponds to the calling user's distinguished name and reject any other modification.

The name of the access right, `Self`, is likely derived from its use as a mechanism for self-group membership. Over time, its use has been expanded to cover a few additional attributes. Microsoft's Active Directory Technical Specification (*MS-ADTS*, available online) refers to it as `RIGHT_DS_WRITE_PROPERTY_EXTENDED`, which is a slightly better description.

We won't perform an example access check for write-validated access because it's the same as the check shown in [Listing 11-35](#) for control access rights; simply change the extended right you query and check that `Self` access is granted. As with `ControlAccess`, it's possible for a non-object ACE to grant `Self` access without having a specific ACE for the write-validated access right.

Note that an administrator can't modify the list of write-validated access rights; this is because the directory server won't know to enforce the restriction. A third-party application can't implement this behavior, either, as its purpose is to limit the changes that can be made to the directory.

Accessing the SELF SID

When I discussed the object type access check in [Chapter 7](#), I also mentioned a principal SID that you can specify to replace the *SELF* SID in an ACE. Active Directory uses the *SELF* SID to grant access to resources based on whether the user making the request is the "self" in question. It extracts the SID to use as this principal SID from the object's `objectSID` attri-

bute, used to store the SID for the user or computer account, as well as the group SID.

For example, if you want to modify a user object in the directory, the server will look up the object's security descriptor and query for the object's `objectSID` attribute. If the attribute is present in the object, the access check will use the value as the principal SID, along with the security descriptor. If the attribute isn't present, no principal SID will be set, and any ACE with the *SELF* SID won't be evaluated. [Listing 11-37](#) shows how to extract the `objectSID` attribute.

```
PS> $computer = Get-ADComputer -Identity $env:COMPUTERNAME
PS> $computer.SID.ToString()
S-1-5-21-1195776225-522706947-2538775957-1104

PS> Get-DsObjectSid -DistinguishedName $computer.DistinguishedName
Name                               Sid
----                               -
MINERAL\GRAPHITE$ S-1-5-21-1195776225-522706947-2538775957-1104
```

Listing 11-37: Getting a computer account's objectSID

There are multiple ways of accessing the attribute. The simplest is to use either the `Get-ADComputer`, `Get-ADUser`, or `Get-ADGroup` command, which will automatically extract the SID. In [Listing 11-37](#), we get the SID for the current computer. Alternatively, if you're using `Get-ADObject`, you can request the `objectSID` attribute to access the property directly.

You can also use a command that comes with the `NtObjectManager` module: `Get-DsObjectSid`, which requires the full distinguished name of the object to query. The main advantage of this command is that it returns a `sid` class you can use in the access check without converting the value into the correct format. You can pass the returned SID to `Get-NtGrantedAccess` in the `Principal` parameter. We'll use it in the worked example at the end of the chapter.

Performing Additional Security Checks

In most cases the access check process grants access to the directory based on the security descriptors assigned to objects, but there are sev-

eral exceptions to this. For example, the directory supports privileges such as `SeRestorePrivilege` and `SeTakeOwnershipPrivilege`, for changing the components of a security descriptor. Let's discuss a few additional non-standard checks.

Adding Workstations to a Domain

In a default domain configuration, the *Authenticated Users* group is granted a special privilege on the domain controller called `SeMachineAccountPrivilege`. This privilege allows any domain user to join a computer to a domain, which, at a low level, means creating a computer object.

When a user tries to create a computer object, the directory server checks whether the caller has `CreateChild` access for the target object. If not, it checks whether they have the `SeMachineAccountPrivilege` privilege. If they do, it allows the creation operation.

However, in the latter case the server limits the attributes the user can set at creation time. For example, the `SeMachineAccountPrivilege` privilege doesn't allow a user to set an arbitrary `NtSecurityDescriptor` attribute; the object must use the default security descriptor. The values for attributes the user is allowed to set, like the username, must also match a fixed pattern, and the security descriptor must use the *Domain Admins* SID as its owner and group SIDs, limiting the user's access to the object after its creation.

An individual user can create only a fixed number of computer accounts. By default, the `ms-DS-MachineAccountQuota` attribute in the root of the directory sets this limit to 10. To enforce this restriction during the creation of a new computer object, the server searches all existing computer objects and checks their `ms-DS-CreatorSID` attribute, which stores the SID of the user who created the object. The server then calculates the number of computers the caller has already added, and if it's over the quota, it rejects the request. However, if the caller has `CreateChild` access, the quota doesn't apply. [Listing 11-38](#) shows how to query these values.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $obj = Get-ADObject $root_dn -Properties 'ms-DS-MachineAccountQuota'
PS> $obj['ms-DS-MachineAccountQuota']
```

10

```

PS> Get-ADComputer -Filter * -Properties 'mS-DS-CreatorSID' | ForEach-Object {
    $creator = $_['mS-DS-CreatorSID']
    if ($creator.Count -gt 0) {
        $sid = Get-NtSid -Sddl $creator[0]
        Write-Host $_.Name, " - ", $sid.Name
    }
}
GRAPHITE - MINERAL\alice
TOPAZ - MINERAL\alice
PYRITE - MINERAL\bob

```

Listing 11-38: Querying the SIDs used to enforce computer account creation quotas

You can create a new computer account using the `New-ADComputer` command, specifying the required attributes. For example, [Listing 11-39](#) creates the computer account `DEMOCOMP` with a known password.

```

PS> $pwd = ConvertTo-SecureString -String "Passw0rd1!!!" -AsPlainText -Force
PS> $name = "DEMOCOMP"
PS> $dnsname = "$name.$((Get-ADDomain).DNSRoot)"
PS> New-ADComputer -Name $name -SAMAccountName "$name$" -DNSHostName $dnsname
    -ServicePrincipalNames "HOST/$name" -AccountPassword $pwd -Enabled $true

```

Listing 11-39: Creating a new computer account in the domain

You can also create an account using the SAM remote service, as shown in [Listing 11-40](#).

```

PS> $sam = Connect-SamServer -ServerName PRIMARYDC
PS> $domain = Get-SamDomain -Server $sam -User
PS> $user = New-SamUser -Domain $domain -Name 'DEMOCOMP$' -AccountType
Workstation
PS> $pwd = ConvertTo-SecureString -String "Passw0rd1!!!" -AsPlainText -Force
PS> $user.SetPassword($pwd, $false)

```

Listing 11-40: Creating a new computer in the domain via the SAM remote service

Servers typically create an account in this way when you join a computer to a domain.

User Delegation Rights

In a default domain configuration, the *Administrators* group is granted a special privilege on the domain controller: the `SeEnableDelegationPrivilege` privilege, which allows users to modify the Kerberos delegation settings. Specifically, it lets them do the following:

- Set the `TrustedForDelegation` user account control flag.
- Set the `TrustedToAuthenticateForDelegation` user account control flag.
- Modify the `msDS-AllowedToDelegateTo` attribute of a user or computer object.

We'll discuss Kerberos delegation and the use of these settings in more detail in [Chapter 14](#).

Protected Objects

The root domain of the directory shares its domain configuration and schema with the entire forest, meaning changes to a user in other domains will eventually be replicated in the root domain. But allowing a child domain to modify the domain configuration or schema is not a good idea, so the server implements a way of protecting objects from being directly modified, deleted, or moved.

Rather than storing this protection as an object attribute or an ACE, the server sets the resource manager control flag in the security descriptor to 1. The technical specification refers to this bit flag as `SECURITY_PRIVATE_OBJECT`. If the object's security descriptor has this flag set and the object is in the schema's or configuration's naming context, then users cannot modify the object unless their owner SID belongs to the same domain as the domain controller on which the modification is being performed.

For example, most objects in the configuration are owned by the *Enterprise Admins* group, a *Universal* group defined in the root domain. So, if an object is protected, only a domain controller in the root domain can modify it directly. [Listing 11-41](#) contains a short script that searches

for protected objects in the configuration naming context by checking the resource manager control flags. No other Windows feature uses these resource manager control flags, as far as I can tell.

```
PS> $conf_nc = (Get-ADRootDSE).configurationNamingContext
PS> Get-ADObject -SearchBase $conf_nc -SearchScope Subtree -Filter * |
ForEach-Object {
    $sd = Get-Win32SecurityDescriptor -Name $_.DistinguishedName -Type Ds
    if ($sd.RmControl -eq 1) {
        $_.DistinguishedName
    }
}
```

Listing 11-41: Finding protected configuration objects

In a default installation of an Active Directory server, [Listing 11-41](#) should output no results, as the directory shouldn't have any protected objects.

This concludes our discussion of access checking, although we'll come back to it in an expansive worked example at the end of the chapter. Next, we'll cover two final Active Directory topics: how user and device claims are stored in the directory, and how group policies are configured.

Claims and Central Access Policies

In the preceding chapters we discussed user and device claims, how tokens store them as security attributes, and how access checks use them. Claims are especially important for enabling central access policies, as we discussed in [Chapter 7](#).

The domain's Active Directory server stores both claims and central access policies, and it can apply these whenever a user authenticates or a computer synchronizes its policy. [Listing 11-42](#) shows how to query the Active Directory server for a claim using the `Get-ADClaimType` PowerShell command, which searches for objects of the schema class `msDS-ClaimType`.

```
PS> Get-ADClaimType -Filter {DisplayName -eq "Country"} |
Format-List ID, ValueType, SourceAttribute, AppliesToClasses
ID           : ad://ext/country
```



```

ValueType      : String
SourceAttribute : CN=Text-Country,CN=Schema,CN=Configuration,...
AppliesToClasses : {CN=User,CN=Schema,CN=Configuration,...}

```

Listing 11-42: Displaying properties of the Country claim

In this example, we find that an administrator configured the `Country` claim when setting up the domain; it isn't available by default. This claim represents the name of the user's country.

We show only a few of the relevant properties of the object. The first is the claim's ID, used for the security attribute in the token; in this case, it's `ad://ext/country`. We also show the value's type, used to determine what security attribute values to add to the token; in this case, it's a string.

The next property is the distinguished name of the schema attribute from which the value is derived. (It's possible for a claim to be derived from other data, such as values on a user's smart card, but sourcing the claim from a schema attribute is the simplest case.) When the user is authenticated, the token will construct the claim based on the attribute value from their user object; if the attribute isn't set, the claim won't be added to the token. An administrator can modify the directory schema to add new attributes from which to derive their own claims, such as a user's security clearance.

Finally, we display the list of schema classes to which this claim applies. In this case, only the `user` schema class appears in the listing. If this list contained the distinguished name of the `computer` class, it would be a device claim, not a user claim, although claims can apply to both users and computers.

Listing 11-43 shows how to display the properties of a central access policy in the directory.

```

PS> $policy = Get-ADCentralAccessPolicy -Identity "Secure Room Policy"
PS> $policy | Format-List PolicyID, Members
PolicyID : S-1-17-3260955821-1180564752-550833841-1617862776
Members  : {CN=Secure Rule,CN=Central Access Rules,CN=Claims...}

PS> $policy.Members | ForEach-Object {Get-ADCentralAccessRule -Identity $_} |

```

```
Format-List Name, ResourceCondition, CurrentAcl
Name           : Secure Rule
ResourceCondition : (@RESOURCE.EnableSecure == 1)
CurrentAcl      : D:(XA;;;FA;;;WD;(@USER.ad://ext/clearance...
```

Listing 11-43: Displaying properties of a central access policy

Administrators deploy central access policies to a domain's computers and servers based on the group policy configuration. This allows them to selectively deploy a policy to a specific subset of systems in the domain. The policy's configuration is stored in the directory, however.

The policy consists of two components: the policy object itself, represented by the `msAuthz-CentralAccessPolicy` schema class, and one or more central access rules, represented by the `msAuthz-CentralAccessRule` schema class.

In [Listing 11-43](#), we first query for a specific central access policy named `Secure Room Policy` using the `Get-ADCentralAccessPolicy` PowerShell command. From the policy we can extract the policy SID, which we use to apply the policy to a resource, as well as a list of the distinguished names of each member rule.

NOTE

The `Get-ADCentralAccessPolicy` command differs from the `Get-CentralAccessPolicy` command I demonstrated in [Chapter 7](#). The former reads all policies from the Active Directory server, whereas the latter shows only the policies configured to be enabled on the local system.

We then use the `Get-ADCentralAccessRule` command to get each of the policy rules. In this example, there is only one rule. We display its name, the resource condition used to determine when the rule is enabled, and the DACL, which determines the level of access a user will be granted on the resource for which the rule is applied. Refer to [Chapters 5](#) and [7](#) for more information about the implementation of central access policies.

Group Policies

On a stand-alone system, the local policy combines information from the LSA policy's configuration with various registry settings that define what applications can do. In a domain network, an administrator can configure a policy for the entire network using *group policies*. Domain-joined computers download these policies on a regular basis (generally, every 90 minutes by default). Computers then merge these group policies with any existing local policy settings to define the computer's overall policy.

Figure 11-6 shows how a domain network configures group policies.

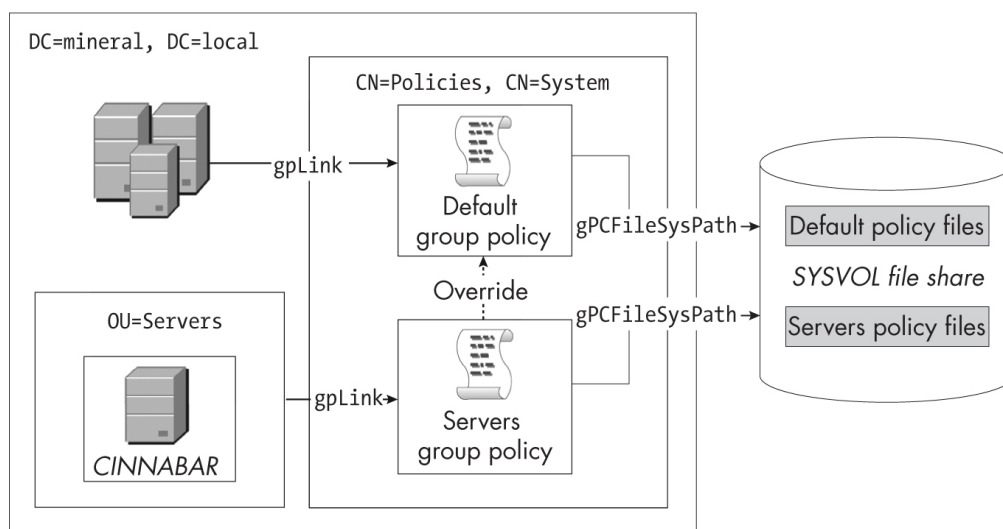


Figure 11-6: The configuration of group policies

The root domain and any organizational unit object can contain the `gpLink` attribute. An *organizational unit* is a directory container that represents some structure in an organization. For example, an administrator could create different organizational units for different offices, then apply different policies for computers within those organizational units.

The `gpLink` attribute contains a list of the domain names belonging to the group policy objects applied to the organizational unit. The group policy objects themselves don't contain the actual policy settings. Instead, the object contains a `gPCFileSysPath` attribute that represents a filepath to a policy configuration file, which contains the settings. This filepath typically points to a special network file share, `SYSVOL`, which contains the configuration files.

What policies to apply depends on where the computer's account object is stored in the directory. For example, in [Figure 11-6](#), the administrator has created the *Servers* organizational unit, then added the *CINNABAR* server account to that container. The organizational unit has the `gpLink` attribute, which links to the Servers Group Policy object.

However, the organizational unit also lives in the root domain, which has its own `gpLink` attribute and assigned policy. When the *CINNABAR* server updates its group policy, it will discover all of these linked group policies in the parent directory hierarchy and use that information to download and apply the policies. The most specific policy takes precedence; for example, for *CINNABAR*, the Servers Group Policy would override conflicting settings in the Default Group Policy. The server will merge any settings that don't conflict when creating the final policy.

In [Listing 11-44](#), we query for group policy objects on the Active Directory server.

```
❶ PS> Get-ADOrganizationalUnit -Filter * -Properties gpLink |
Format-List Name, LinkedGroupPolicyObjects
Name                : Domain Controllers
LinkedGroupPolicyObjects : {CN={6AC1786C-016F-11D2-945F-00C04fB984F9},...}

❷ PS> $policy = Get-ADObject -Filter {
    ObjectClass -eq "groupPolicyContainer"
} -Properties *
PS> $policy | Format-List displayName, gPCFileSysPath
displayName      : Default Domain Policy
gPCFileSysPath   : \\mineral.local\sysvol\mineral.local\Policies\{31B2F340-...}

displayName      : Default Domain Controllers Policy
gPCFileSysPath   : \\mineral.local\sysvol\mineral.local\Policies\{6AC1786C-...}

displayName      : Default Servers Domain Policy
gPCFileSysPath   : \\mineral.local\sysvol\mineral.local\Policies\{6B108F70-...}

❸ PS> ls $policy[0].gPCFileSysPath
Directory: \\mineral.local\sysvol\mineral.local\Policies\{31B2F340-016D-...}

Mode                LastWriteTime         Length Name
----                -
d-----           3/12/2023  12:56 PM             Adm
```

```

d-----          3/12/2023    1:02 PM                MACHINE
d-----          4/6/2023    8:18 PM                USER
-a-----          4/6/2023    8:24 PM                22  GPT.INI

❷ PS> $dc_policy = $policy |
Where-Object DisplayName -eq "Default Domain Controllers Policy"
PS> $dc_path = $dc_policy.gPCFileSysPath
PS> Get-Content "$dc_path\MACHINE\Microsoft\Windows NT\SecEdit\GptTmpl.inf" |
Select-String "SeEnableDelegationPrivilege", "SeMachineAccountPrivilege"
❸ SeMachineAccountPrivilege = *S-1-5-11
SeEnableDelegationPrivilege = *S-1-5-32-544

```

Listing 11-44: Finding group policy objects

First, we query for organizational unit objects in the directory using the `Get-ADOrganizationalUnit` command and request the `gplink` attribute ❶. We display the name and the list of group policy objects for each organizational unit.

We could now take the group policy object's distinguished names from the `gplink` attribute and manually look up each one. Instead, let's simply search for all objects of class `groupPolicyContainer` using the `Get-ADObject` PowerShell command ❷. This shows us the name of each policy object, as well as the path to the real policy store on the *SYSVOL* file server.

We can also list the contents of the policy directory on the file server ❸. Depending on how complex the policy is, the file share might contain many different files. A group policy can apply to a particular machine, as well as on a per-user basis, which is why there are separate *MACHINE* and *USER* directories.

We won't discuss the configuration of group policies any further, but I recommend inspecting the files contained in the file share during your security research. Group policies can contain a wealth of information related to the configuration of computers and users in the domain. Sometimes this policy configuration includes shared passwords for user accounts or private key material. Because any user on the network can access the *SYSVOL* share, an attacker could extract this information to gain additional privileges on the network.

As a minor example of information leakage, you could determine which SIDs would be granted the two special privileges, `SeMachineAccountPrivilege` and `SeEnableDelegationPrivilege`, on a domain controller. The group policy assigned to the domain controller typically stores this privilege assignment information in the *GptTmpl.inf* file, which any user in the domain can access. (The LSA domain policy remote service discussed in [Chapter 10](#) can also provide this information, but it requires administrator privileges.)

In [Listing 11-44](#), we retrieve the Default Domain Controllers Policy ④, the only policy applied in our simple environment. We then extract the privileges from the file using a simple string selection. In this example, we find the default configuration: *Authenticated Users* is granted `SeMachineAccountPrivilege`, and *BUILTIN\Administrators* is granted `SeEnableDelegationPrivilege` ⑤.

Worked Example

In this chapter's single worked example, we'll walk through a script that checks a user's access to every object we can find in the local Active Directory server. This process is quite involved, so I've broken it into multiple sections.

Building the Authorization Context

Throughout this chapter, we've been using the `Get-NtGrantedAccess` command to run the access check for a security descriptor. This command is fine for testing purposes, but it causes a subtle problem when used to check real-world security descriptors in the Active Directory server.

The command uses the `NtAccessCheck` system call, which uses a `Token` object to represent the user's identity. However, the token's group membership is based on the local system's LSA user configuration, and the domain controller is unlikely to use the same groups. For example, many security descriptors in the directory grant full access to the *BUILTIN\Administrators* group, but these local administrators won't necessarily also be administrators on the domain controller.

We need a way of running an access check using the groups from the domain controller. One option is to run the access check on the domain con-

troller itself. However, that only works if we have full control over the network, which is best avoided. A second option would be to manually create a token with the necessary groups, but this would still require elevated local privileges. Finally, we could use our own implementation of the access check, such as the one we built in [Chapter 7](#), but this risks introducing incorrect behavior.

We do have another option: Windows provides the `AuthZ` (authorization) API, which has a function called `AuthZAccessCheck` that we can use to perform an access check based on a constructed authorization context rather than a token. This API runs entirely in user mode, and the authorization context for a user can contain any groups the caller likes. If you don't want to enable auditing, the APIs also work without any elevated privileges.

A big advantage of using the `AuthZ` API over a custom access check implementation is that it shares code with the kernel's own access check implementation, and therefore, it should be correct. As a bonus, it's also the same API used by the Active Directory server to perform access checks, so its results should match the server's when given the correct authorization context.

We can build an authorization context for a domain user based only on information that we can extract from the domain without administrator privileges. [Listing 11-45](#) shows how to build the authorization context.

```
❶ PS> function Add-Member($Set, $MemberOf) {  
    foreach($name in $MemberOf) {  
        if ($Set.Add($name)) {  
            $group = Get-ADGroup $name -Properties MemberOf  
            Add-Member $Set $group.MemberOf  
        }  
    }  
}  
  
❷ PS> function Get-UserGroupMembership($User) {  
    $groups = [System.Collections.Generic.HashSet[string]]::new(  
        [System.StringComparer]::OrdinalIgnoreCase  
    )  
    ❸ Add-Member $groups $User.PrimaryGroup  
    Add-Member $groups $User.MemberOf
```

```

❷ $auth_users = Get-ADObject -Filter {
    ObjectClass -eq "foreignSecurityPrincipal" -and Name -eq "S-1-5-11"
} -Properties memberOf
Add-Member $groups $auth_users.MemberOf
❸ $groups | ForEach-Object {Get-DsObjectSid $_}
}

PS> function Get-AuthContext($username) {
    ❹ $user = Get-ADUser -Identity $username -Properties memberOf, primaryGroup
    -ErrorAction Continue
    if ($null -eq $user) {
        $user = Get-ADComputer -Identity $username -Properties memberOf,
primaryGroup
    }
    $sids = Get-UserGroupMembership $user

    ❺ $rm = New-AuthZResourceManager
    ❻ $ctx = New-AuthZContext -ResourceManager $rm -Sid $user.SID.Value
-Flags SkipTokenGroups
    ❼ Add-AuthZSid $ctx -KnownSid World
    Add-AuthZSid $ctx -KnownSid AuthenticatedUsers
    Add-AuthZSid $ctx -Sid $sids
    $rm.Dispose()
    $ctx
}

❽ PS> $ctx = Get-AuthContext "alice"
PS> $ctx.Groups
Name                               Attributes
----
Everyone                           Enabled
NT AUTHORITY\Authenticated Users   Enabled
MINERAL\Domain Users               Enabled
BUILTIN\Users                      Enabled
BUILTIN\Pre-Windows 2000 Compatible Access Enabled

```

Listing 11-45: Building an authorization context for the access check

In the directory, user and group objects have a `memberOf` attribute that lists the distinguished names of the group objects that the user or group is a member of. We can use this list to recursively inspect the directory to find all groups. This is what the `Add-Member` function is doing ❶.

We then define a function to get a list of member SIDs from a user object ❷. We need to add the root groups, which include the user's primary group ❸ and groups referenced by the `memberOf` attribute. We also need to add groups from SIDs that are outside the domain. These are stored as foreign security principals. In the example, we find the entry for *Authenticated Users*, a group that all users are a member of, and add its group memberships ❹. We now have a list of distinguished names for group objects, which we convert to a list of SIDs that we can add to the authorization context ❺.

Next, we build the authorization context itself. We start by querying for the user object ❻; if that fails, we check for a computer object and get the list of SIDs the account is a member of. Then we create an `AuthZ` resource manager ❼, which (as its name suggests) is used to manage resources. For example, we can use it to cache access checks between contexts.

We create the authorization context using the `New-AuthZContext` command ❽. We need to specify the `SkipTokenGroups` flag when creating the context so that only the user's SID gets added to it. Otherwise, the context will contain the list of local groups, which defeats the purpose of gathering the groups on the domain controller.

We then use the `Add-AuthZSid` command to add the group SIDs to the context ❾, making sure to include the default *World* and *Authenticated Users* groups. Finally, we test the behavior of the functions for the user *alice* ❿, printing out the list of domain groups the user is a member of on the domain controller.

THE REMOTE ACCESS CHECK PROTOCOL

The `AuthZ` API supports another mechanism for running an access check with the correct group list, but without running code directly on the domain controller. Computers on a domain, including the domain controller, expose a remote access check network protocol that you can connect to when creating the resource manager.

Normal users on the domain won't be able to call the protocol, which requires the calling user to be a member of either the *BUILTIN\Administrators* or

BUILTIN Access Control Assistance Operators group on the domain controller, making it somewhat less useful. However, you might be a member of one of these groups without even realizing it, so it's worth trying to connect to the service and perform an access check. The following commands create an authorization context with a connection to the *PRIMARYDC* domain controller:

```
PS> $rm = New-AuthZResourceManager -Server PRIMARYDC.mineral.local
PS> $ctx = New-AuthZContext -ResourceManager $rm -Sid (Get-NtSid)
PS> $ctx.User
```

Name	Attributes
----	-----
MINERAL\alice	None


```
PS> $ctx.Groups
```

Name	Attributes
----	-----
MINERAL\Domain Users	Mandatory, EnabledByDefault, Enabled
Everyone	Mandatory, EnabledByDefault, Enabled
BUILTIN\Access Control Assistance...	Mandatory, EnabledByDefault, Enabled

--snip--

These commands could replace the entirety of [Listing 11-45](#). To use the remote access check protocol, we specify the DNS name of the domain controller using the *Server* parameter of the *New-AuthZResourceManager* command. We then create the *AuthZ* context with the *SID* of the user. We don't need to specify any flags, as the service will base the group list on the server running the remote access check protocol (in this case, the domain controller). We can verify the assigned user and groups to confirm that their values are based on the domain controller's local group assignment.

Gathering Object Information

With the authorization context in hand, we can begin the access check. We'll use the *Get-AuthZGrantedAccess* command, which works almost the same as *Get-NtGrantedAccess* but relies on the context we've created. We'll start by gathering information about the object we want to check. We need the following details:

- The security descriptor of the object

- The object SID, if present, for the principal SID
- All schema classes, including auxiliary and child classes
- Allowed schema attributes and associated property sets
- Applicable control and write-validated access rights

Listing 11-46 implements the `Get-ObjectInformation` function, which gathers this information about an object based on its distinguished name.

```
PS> function Get-ObjectInformation($Name) {
    $schema_class = Get-DsObjectSchemaClass $Name
    $sid = Get-DsObjectSid $Name
    $all_classes = Get-DsSchemaClass $schema_class.Name -Recurse -IncludeAuxiliar
    $attrs = $all_classes.Attributes | Get-DsSchemaAttribute |
Sort Name -Unique
    $infs = Get-DsSchemaClass $schema_class.Name -Inferior
    $rights = $all_classes | ForEach-Object {Get-DsExtendedRight -SchemaClass $_}
Sort Name -Unique
    [PSCustomObject]@{
        Name=$Name
        SecurityDescriptor=Get-Win32SecurityDescriptor -Name $Name -Type Ds
        SchemaClass=Get-DsObjectSchemaClass $Name
        Principal=$sid
        Attributes=$attrs
        Inferiors=$infs
        PropertySets=$rights | Where-Object IsPropertySet
        ControlRight=$rights | Where-Object IsControl
        ValidatedWrite=$rights | Where-Object IsValidatedWrite
    }
}
```

Listing 11-46: Implementing the `Get-ObjectInformation` function

We can test the function by passing it the distinguished name of the object for which we want the information, as shown in **Listing 11-47**.

```
PS> $dn_root = (Get-ADRootDSE).defaultNamingContext
PS> Get-ObjectInformation $dn_root
Name                : DC=mineral,DC=local
SchemaClass         : domainDNS
Principal           : S-1-5-21-146569114-2614008856-3334332795
Attributes          : {adminDescription, adminDisplayName...}
Inferiors           : {device, samServer, ipNetwork, organizationalUnit...}
```

```

PropertySets      : {Domain-Other-Parameters, Domain-Password}
ControlRight      : {Add-GUID, Change-PDC, Create-Inbound-Forest-Trust...}
ValidatedWrite    :
SecurityDescriptor : 0:BAG:BAD:AI(OA;CIIO;RP;4c164200-20c0-11d0-...

```

Listing 11-47: Gathering object information

In this example, we request the information for the root domain object. You could cache most of the returned information about the schema class, as only the security descriptor and object SID typically change between objects. However, for simplicity, we'll gather the information for every request.

Running the Access Check

We now have everything we need to perform a maximum access check for an object. However, it's not as simple as passing the security descriptor and the authorization context to the `AuthZ` access check API and calling it a day. We must separately handle each type of resource (such as classes, attributes, control access rights, and write-validated access rights) to make sure we capture the maximum allowed access.

Listing 11-48 contains the functions to run the access check process. For simplicity, we'll focus on capturing access rights that could result in a modification of the object. However, you could easily modify the functions to capture read access, as well.

```

❶ PS> function Test-Access($Ctx, $Obj, $ObjTree, $Access) {
    Get-AuthZGrantedAccess -Context $ctx -ObjectType $ObjTree
    -SecurityDescriptor $Obj.SecurityDescriptor -Principal $Obj.Principal
    -Access $Access | Where-Object IsSuccess
}

PS> function Get-PropertyObjTree($Obj) {
    $obj_tree = New-ObjectTypeTree $Obj.SchemaClass
    ❷ foreach($prop_set in $Obj.PropertySets) {
        Add-ObjectTypeTree $obj_tree $prop_set
    }

    ❸ $fake_set = Add-ObjectTypeTree $obj_tree -PassThru
    -ObjectType "771727b1-31b8-4cdf-ae62-4fe39fadf89e"
}

```

```

foreach($attr in $Obj.Attributes) {
    if (-not $attr.IsPropertySet) {
        Add-ObjectTypeTree $fake_set $attr
    }
}
$obj_tree
}

PS> function Get-AccessCheckResult($Ctx, $Name) {
    try {
        ④ $obj = Get-ObjectInformation $Name
        $access = Test-Access $ctx $obj $obj.SchemaClass "MaximumAllowed" |
        Select-Object -ExpandProperty SpecificGrantedAccess

        ⑤ $obj_tree = Get-PropertyObjTree $obj
        $write_attr = Test-Access $ctx $obj $obj_tree "WriteProp"
        $write_sets = $write_attr | Where-Object Level -eq 1 |
        Select-Object -ExpandProperty Name
        $write_attr = $write_attr | Where-Object Level -eq 2 |
        Select-Object -ExpandProperty Name

        ⑥ $obj_tree = New-ObjectTypeTree
        -ObjectType "771727b1-31b8-4cdf-ae62-4fe39f89e"
        $obj.Inferiors | Add-ObjectTypeTree -Tree $obj_tree

        $create_child = Test-Access $ctx $obj $obj_tree "CreateChild" |
        Where-Object Level -eq 1 | Select-Object -ExpandProperty Name
        $delete_child = Test-Access $ctx $obj $obj_tree "DeleteChild" |
        Where-Object Level -eq 1 | Select-Object -ExpandProperty Name

        ⑦ $control = if ($obj.ControlRight.Count -gt 0) {
            $obj_tree = New-ObjectTypeTree -SchemaObject $obj.SchemaClass
            $obj.ControlRight | Add-ObjectTypeTree $obj_tree
            Test-Access $ctx $obj $obj_tree "ControlAccess" |
            Where-Object Level -eq 1 | Select-Object -ExpandProperty Name
        }

        ⑧ $write_valid = if ($obj.ValidatedWrite.Count -gt 0) {
            $obj_tree = New-ObjectTypeTree -SchemaObject $obj.SchemaClass
            $obj.ValidatedWrite | Add-ObjectTypeTree $obj_tree
            Test-Access $ctx $obj $obj_tree "Self" |
            Where-Object Level -eq 1 | Select-Object -ExpandProperty Name
        }

        ⑨ [PSCustomObject]@{

```

```

        Name=$Obj.Name
        Access=$access
        WriteAttributes=$write_attr
        WritePropertySets=$write_sets
        CreateChild=$create_child
        DeleteChild=$delete_child
        Control=$control
        WriteValidated=$write_valid
    }
} catch {
    Write-Error "Error testing $Name - $_"
}
}

```

Listing 11-48: Running the object access check

We start by defining a few helper functions. The first, `Test-Access`, runs the access check based on the authorization context, the security descriptor, the object type tree, and a desired access mask ❶. The access check returns a list of results for each checked object type. We're interested only in the ones that succeeded, granting some access.

The next helper, `Get-PropertyObjTree`, builds the object type tree used for checking property sets and attributes. The root of the tree is the object's schema class identifier. From there, we first populate all available property sets ❷. We then add all remaining attributes that aren't already in a property set by placing them into a separate dummy set ❸.

We can now move on to the multiple access check functions. First we get the information for an object based on its distinguished name ❹. We then get the maximum granted access for the object, with only the object schema class identifier as the object type. This gives us an idea of the basic rights the user will be granted, such as the ability to delete the object or modify its security descriptor.

Next, we build the tree for the property sets and attributes ❺ and run the access check using the `Test-Access` function. We're interested only in results that grant `WriteProp` access (most objects let any user read their attributes, so this information is less interesting). We split the access check results into writable property sets and writable individual attributes.

We now focus on the child classes by building the object type tree from the schema class identifier ⑥. Even though the directory server would check a single class at a time, we'll perform all the checks in one go. We run two access checks, one for `CreateChild` access and one for `DeleteChild` access.

One thing to note is that we use the dummy identifier as the root object type. If we instead used the schema class identifier for the object, the access granted to that class would propagate to all the children, potentially giving us the wrong result. Using an identifier that isn't a real schema class should enable us to avoid this outcome.

We run a similar access check for control access rights ⑦ and write-validated access rights ⑧, requesting `ControlAccess` and `Self`, respectively. Finally, we package all the results into a custom object to return to the caller ⑨.

Listing 11-49 demonstrates calling the `Get-AccessCheckResult` function for an Active Directory object.

```
PS> $dn = "CN=GRAPHITE,CN=Computers,DC=mineral,DC=local"
PS> $ctx = Get-AuthContext 'alice' ①
PS> Get-AccessCheckResult $ctx $dn ②
Name           : CN=GRAPHITE,CN=Computers,DC=mineral,DC=local
Access         : List, ReadProp, ListObject, ControlAccess, ReadControl
WriteAttributes : {displayName, sAMAccountName, description, accountExpires...}
WritePropertySets : {User-Account-Restrictions, User-Logon}
CreateChild    :
DeleteChild    :
Control        : {Allowed-To-Authenticate, Receive-As, Send-As,...}
WriteValidated : Validated-SPN

PS> $ctx = Get-AuthContext $dn
PS> Get-AccessCheckResult $ctx $dn ③
Name           : CN=GRAPHITE,CN=Computers,DC=mineral,DC=local
Access         : CreateChild, DeleteChild, List, ReadProp, ListObject,...
WriteAttributes : {streetAddress, homePostalAddress, assistant, info...}
WritePropertySets : {Personal-Information, Private-Information}
CreateChild    : {msFVE-RecoveryInformation, ms-net-ieee-80211-...}
DeleteChild    : {msFVE-RecoveryInformation, ms-net-ieee-80211-...}
```

```
Control      : User-Change-Password
WriteValidated : {DS-Validated-Write-Computer, Validated-SPN}
```

Listing 11-49: Testing the Get-AccessCheckResult function

In this example I've used the *GRAPHITE* computer object, but you can change this distinguished name to that of any object you want to check in the directory. We first need to get the authentication context for the user (here, *alice*) ❶. This user created the *GRAPHITE* object and therefore has some special access other users don't have.

Next, we run the access check and display the results to the console ❷. You can see in the `Access` property that `ControlAccess` has been granted generally. This means that *alice* can use any control access right unless it is explicitly denied through an ACE (a `Denied` ACE also applies whenever a user or computer is marked as "User cannot change password," blocking the `User-Change-Password` control access right).

We can see that the user has some writable attributes and property sets but can't create or delete any child objects. We additionally see the list of granted control and write-validated access rights. The control access rights are granted based on the top-level granted access, but the `Validated-SPN` access right must have been granted explicitly.

Next, we repeat the check using the computer account ❸. If you compare the output with that for *alice*, you'll notice several differences. First, the attributes and property sets that the user can write to have changed. More importantly, the computer account can create and delete any child object. The computer account also has fewer control access rights, but more write-validated access rights.

You can enumerate all objects in the local Active Directory server using the `Get-ADObject` command, then pass each distinguished name to the `Get-AccessCheckResult` function to enumerate writable access across the entire directory.

This concludes our worked example. Hopefully, it has given you a better understanding of the nuts and bolts of the Active Directory server access check process. If you'd like to explore an existing implementation of the

access check, the `NtObjectManager` module provides the `Get-AccessibleDsObject` command, which checks for read access in addition to write access and caches domain information to improve performance. You can use it to run a full recursive scan of the Active Directory server for the current user with the command shown in [Listing 11-50](#).

```
PS> Get-AccessibleDsObject -NamingContext Default -Recurse
Name      ObjectClass  Username      Modifiable Controllable
-----
domain    domainDNS    MINERAL\alice False       True
Builtin    builtinDomain MINERAL\alice False       False
Computers container    MINERAL\alice False       False
--snip--
```

Listing 11-50: Performing an access check

The tabular output indicates whether the user can modify each type of object, such as by changing its attributes or creating a child object, and whether any control access rights have been granted to the user for that object.

Wrapping Up

We began this long chapter with a high-level overview of the information stored in Active Directory, such as the users and groups that are part of the domain, and we inspected the directory's configuration from PowerShell using the Remote Server Administration Tools.

We then dug into the Active Directory server at a lower level, starting with its schema, which defines the structure of the directory. The Active Directory server consists of hierarchical objects that can contain named values called attributes. Each object and attribute has a schema representation that defines what it can contain.

Next, we discussed how the Active Directory server secures objects through a mandatory security descriptor attribute. We looked at examples of querying the security descriptors of existing objects, as well as how to create security descriptors for new objects. We also saw how to assign security descriptors to existing objects.

Once we understood how an object's security descriptor is configured, we explored how the directory server determines what access a user has to an object and its attributes. This access check process uses unique identifiers taken from the schema representation to build object type trees. These make the access check granular, able to grant a user access to only a specific attribute without requiring thousands of hardcoded checks.

The Active Directory configuration also contains two special types of access rights: control access rights and write-validated access writes. These allow users to perform special operations on an object, such as changing a user's password; they also prevent a user from modifying certain attribute values without confirmation from the server.

The access check process contains a few exceptions too. For example, a user can be granted the `SeMachineAccountPrivilege` privilege, which allows them to create computer objects even if no directory object grants them the necessary permission. This allows users to join their computer to a domain without needing an administrator account. However, the directory server limits what the user can do with the new computer account, to mitigate the risk of compromise.

Lastly, we went through a very quick overview of how a domain configures group policies through links to external network filesystems. We noted that this design could leak information about the configuration of users on a domain controller to users without administrative access.

We'll return to the topic of Active Directory when we discuss Kerberos authentication in [Chapter 14](#). Keep in mind that real-world deployments of Windows domains can be extremely complex, with many more security nuances than covered here. If you'd like to know more about how Active Directory functions and the many security edge cases it presents, consult Microsoft's technical specification for Active Directory (*MS-ADTS*).

In the next chapter, we're going to delve into how interactive authentication is implemented on Windows. This authentication allows you to log in to a desktop and use the computer's user interface.

